



UNIVERSIDADE
FEDERAL DO CEARÁ

Relatório Final

Manutenção De Software - 01 - 2026.1

Profa. Carla Ilane Moreira Bezerra

Autores

João Eudes Silva Filho

553166

Sumário

Sumário	1
1. Caracterização do Projeto	3
2. Métricas Antes da Refatoração	3
2.1 Radon Antes da Refatoração	3
2.1.1 Complexidade Ciclomática por Arquivo Antes da Refatoração	3
2.1.2 Complexidade Ciclomática por Função Antes da Refatoração	4
2.1.3 Índice de Manutenibilidade Antes da Refatoração	4
2.1.4 Métricas brutas Antes da Refatoração	5
2.2 CodeCarbon Antes da Refatoração	5
2.3 Pylint Antes da Refatoração	5
2.3.1 Distribuição do Smells Antes da Refatoração	5
2.3.2 Distribuição dos Problemas de Código Antes da Refatoração	6
2.3.3 Arquivos mais Críticos Antes da Refatoração	6
2.3.4 Score Projeto Antes da Refatoração	7
2.4 Pytest Antes da Refatoração	7
2.4.1 Cobertura de testes Antes da Refatoração	7
2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração	9
3. Processo de Refatoração	9
3.1 too-many-statements	9
3.1.1 Ocorrência Número 1	9
3.1.2 Ocorrência Número 2	10
3.2 too-many-instance-attributes	11
3.3 too-many-branches	11
4. Métricas Após a Refatoração	11
4.1 Radon Após a Refatoração	11
4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração	11
4.1.2 Complexidade Ciclomática por Função Após a Refatoração	11
4.1.3 Índice de Manutenibilidade Após a Refatoração	12
4.1.4 Métricas Brutas Após a Refatoração	12
4.2 CodeCarbon Após a Refatoração	12
4.3 Pylint Após a Refatoração	13
4.3.1 Distribuição do Smells Após a Refatoração	13
4.3.2 Distribuição dos Problemas de Código Após a Refatoração	13
4.3.3 Arquivos mais Críticos Após a Refatoração	13
4.3.4 Score Projeto Após a Refatoração	13
4.4 Pytest Após a Refatoração	13
4.4.1 Cobertura de testes Após a Refatoração	13
4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração	13

5. Comparação dos Resultados	14
6. Percepções sobre a Prática	14
7. Links	14

1. Caracterização do Projeto

Nome Projeto: IMBENS (Class-Imbalance Benchmark and Solutions)

O IMBENS é uma biblioteca Python extensível de código aberto voltada para a implementação, avaliação e comparação rápida de soluções de aprendizado para problemas de classificação com classes desbalanceadas. As principais capacidades incluem:

- **Amplo repertório de algoritmos:** Suporte a mais de 30 métodos, incluindo técnicas de *under-sampling* (baseadas em seleção ou geração), *over-sampling* (como SMOTE e variantes), aprendizado sensível a custos (*cost-sensitive learning*) e métodos de *ensemble* (como SMOTEBagging, RUSBoost e SelfPacedEnsemble).
- **Integração e Compatibilidade:** Construído com base nos princípios de design do scikit-learn, garantindo compatibilidade total com o ecossistema de aprendizado de máquina em Python.
- **Ferramentas de análise:** Oferece visualizadores integrados para comparação de modelos, geração de curvas de performance e matrizes de confusão.
- **Eficiência e Monitoramento:** Otimizado com processamento paralelo (via joblib) e suporte a logs de treinamento interativos e customizáveis.

O propósito do IMBENS é fornecer um ambiente unificado e de fácil utilização para lidar com o problema do "desbalanceamento de classes" (ou problema da cauda longa), que é comum em aplicações práticas como detecção de fraudes, diagnósticos médicos raros e análise genética. O projeto resolve uma dificuldade crítica na ciência de dados: a queda no desempenho preditivo de algoritmos tradicionais quando as classes não estão representadas de forma igual. Ao facilitar a aplicação, o benchmarking e o monitoramento dessas técnicas, o IMBENS permite que pesquisadores e desenvolvedores criem modelos mais imparciais e precisos sem a necessidade de implementar manualmente cada técnica de rebalanceamento complexa.

2. Métricas Antes da Refatoração

Nesta seção você vai apresentar as principais métricas obtidas antes da refatoração para cada ferramenta, não precisa ser todas as métricas. Confira um exemplo abaixo:

2.1 Radon Antes da Refatoração

2.1.1 Complexidade Ciclômática por Arquivo Antes da Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
imbens/pipeline.py	8	5.87	16	47	C	_validate_steps
imbens/datasets/_openml.py	2	12.0	16	24	C	_fetch_openml_data
imbens/datasets/_zenodo.py	1	15.0	15	15	C	_fetch_openml_data
imbens/ensemble/_boost.py	15	4.6	14	69	C	_evaluate
imbens/ensemble/base.py	12	3.5	14	42	C	_evaluate

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
imbens/exceptions.py	1	1.0	1	1	A	raise_isinstance_error
imbens/base.py	1	1.0	1	1	A	make_imbalance
imbens/datasets/init.py	1	1.0	1	1	A	(init)
imbens/_version.py	1	1.0	1	1	A	(version)
imbens/datasets/tests/init.py	1	1.0	1	1	A	(test_init)

2.1.2 Complexidade Ciclômática por Função Antes da Refatoração

Apresente as funções do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
imbens/pipeline.py	method	_validate_steps	C	16
imbens/datasets/_openml.py	function	i_fetch_openml_data	C	16

imbens/ensemble/base.py	method	_evaluate	C	14
imbens/datasets/_zenodo.py	function	fetch_zenodo_datasets	C	15
imbens/pipeline.py	method	_fit	C	11

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
imbens/exceptions.py	function	raise_isinstance_error	A	1
imbens/pipeline.py	method	fit_predict	A	1
imbens/datasets/_imbalance.py	function	generate_imbalance_data	A	1
imbens/base.py	method	init	A	1
imbens/ensemble/base.py	method	init	A	1

2.1.3 Índice de Manutenibilidade Antes da Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela antes da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	mi	rank_mi
imbens/ensemble/_boost.py	33.78	A
imbens/ensemble/base.py	40.46	A
imbens/ensemble/_bagging.py	45.14	A
imbens/datasets/_openml.py	46.27	A
imbens/pipeline.py	48.45	A

Tabela de melhores métricas

arquivo	mi	rank_mi
imbens/base.py	100.0	A
imbens/exceptions.py	100.0	A
imbens/_version.py	100.0	A
imbens/init.py	100.0	A

imbens/datasets/init.py	100.0	A
-------------------------	-------	---

2.1.4 Métricas brutas Antes da Refatoração

Essas métricas podem ser encontradas no arquivo *raw_por_arquivo_e_total_antes.csv*. Traga apenas o total, que se encontra no final do csv.

loc total	lloc total	sloc total	comments total	multi
5945	1854	2695	370	1293

2.2 CodeCarbon Antes da Refatoração

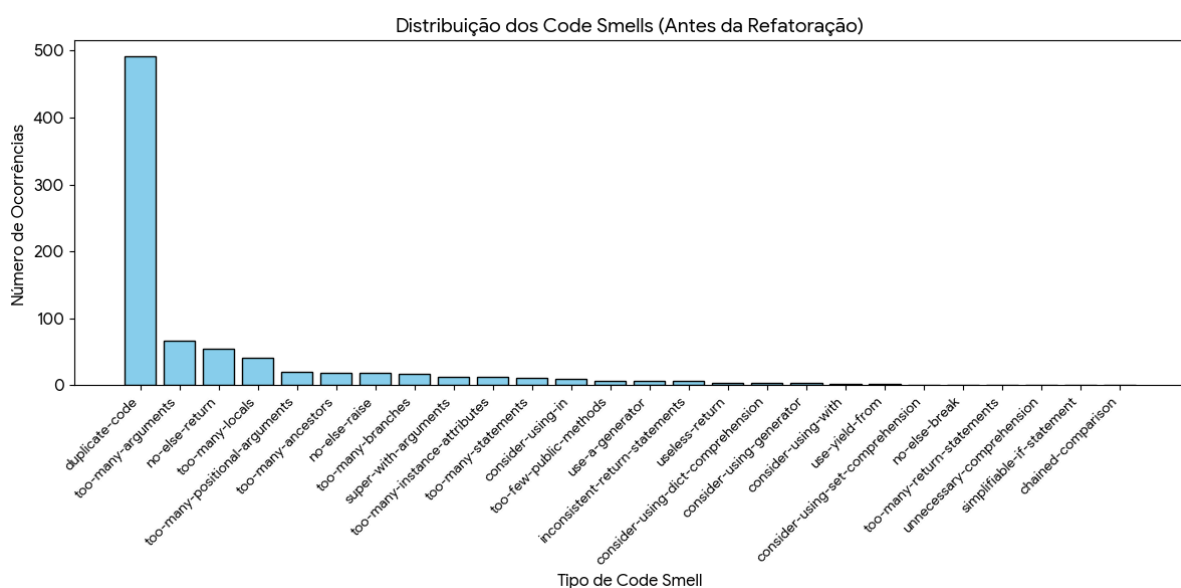
Preencha a tabela abaixo com as métricas ambientais antes da refatoração.

duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
105.9188	9.236169783848518e-05	8.720047861351874e-07	9.671661975782338e-05	3.016372132349767e-04	9.391314444589252e-04

2.3 Pylint Antes da Refatoração

2.3.1 Distribuição do Smells Antes da Refatoração

A partir do JSON *pylint_ranking_smells_antes.json* gere um gráfico para representar a distribuição dos code smells antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



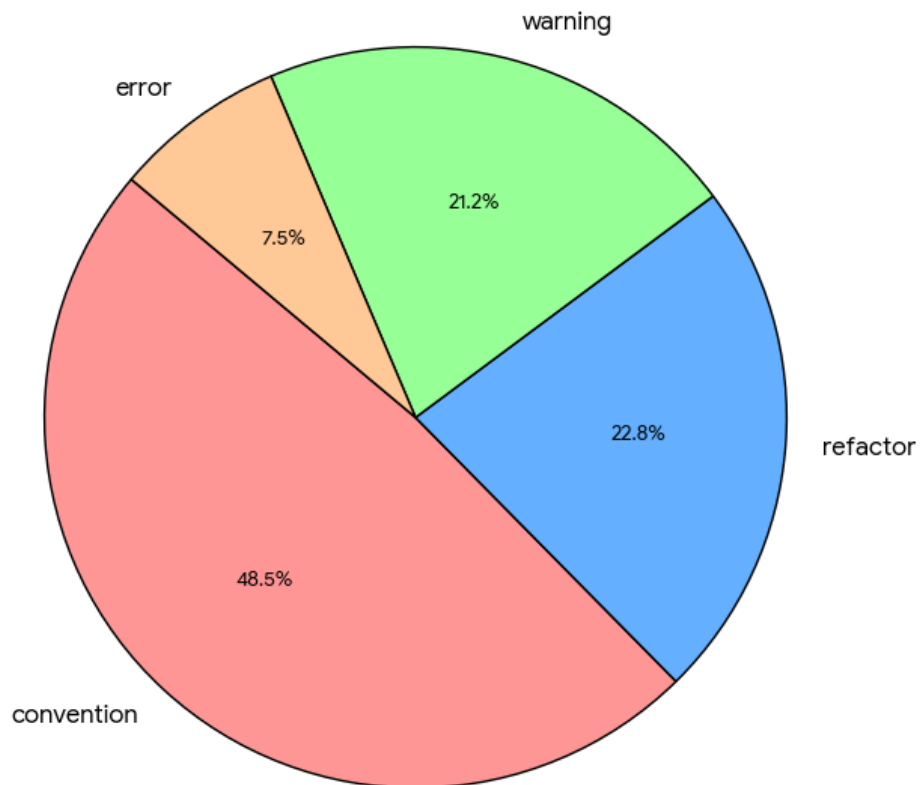
Distribuição dos Code Smells (Antes da Refatoração)

- **duplicate-code**: 491 ocorrências
- **too-many-arguments**: 66 ocorrências
- **no-else-return**: 55 ocorrências
- **too-many-locals**: 41 ocorrências
- **too-many-positional-arguments**: 20 ocorrências
- **too-many-ancestors**: 19 ocorrências
- **no-else-raise**: 18 ocorrências
- **too-many-branches**: 17 ocorrências
- **super-with-arguments**: 13 ocorrências
- **too-many-instance-attributes**: 12 ocorrências
- **too-many-statements**: 11 ocorrências
- **consider-using-in**: 9 ocorrências
- **too-few-public-methods**: 7 ocorrências
- **inconsistent-return-statements**: 6 ocorrências
- **use-a-generator**: 6 ocorrências
- **useless-return**: 3 ocorrências
- **consider-using-dict-comprehension**: 3 ocorrências

2.3.2 Distribuição dos Problemas de Código Antes da Refatoração

A partir do JSON `pylint_distribuicao_categorias_antes.json` gere um gráfico para representar a distribuição entre as 4 categorias de problemas de código antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.

Distribuição de Categorias de Problemas (Antes da Refatoração)

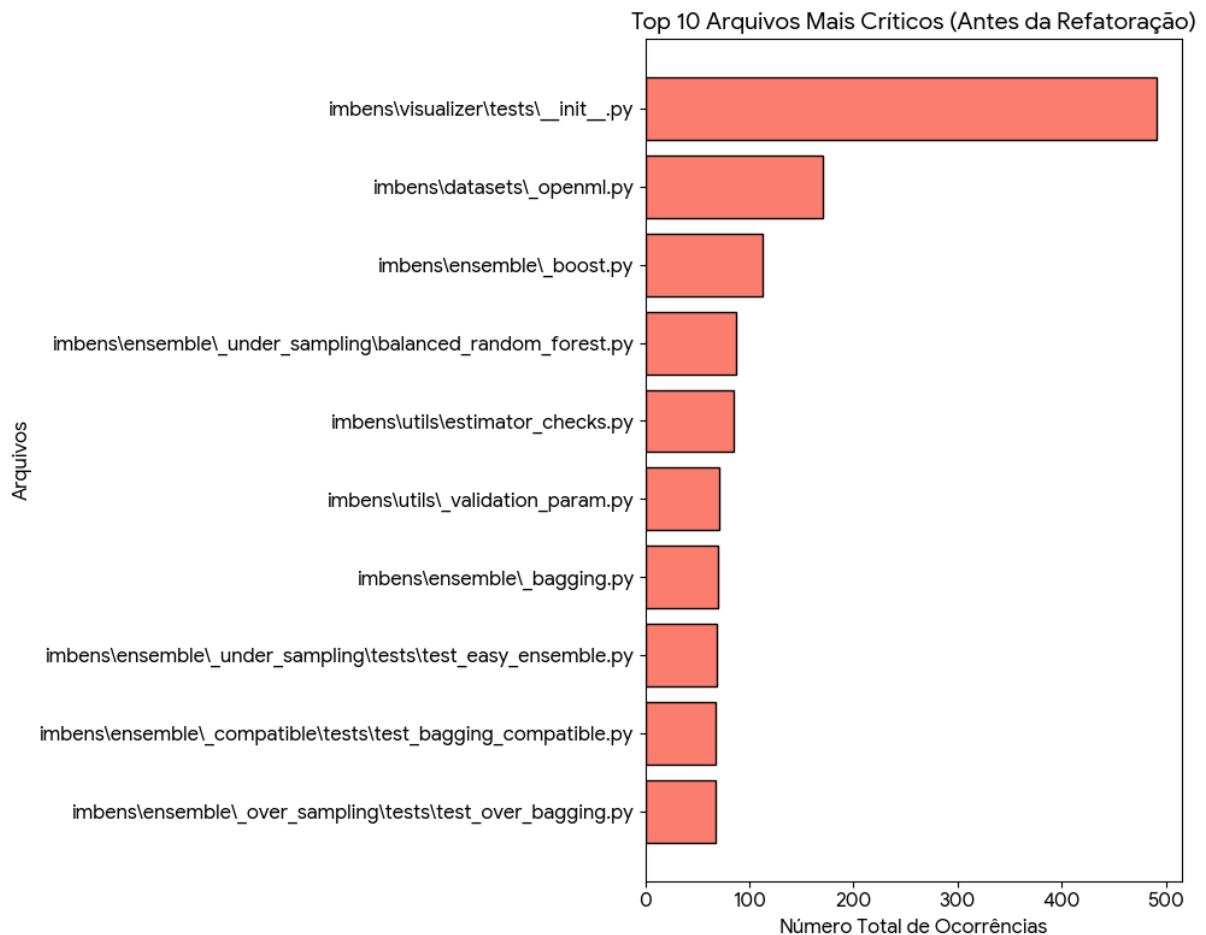


Resumo por Categorias (Antes da Refatoração)

- Convention: 1722 ocorrências (48,49%)
- Refactor: 810 ocorrências (22,81%)
- Warning: 752 ocorrências (21,18%)
- Error: 267 ocorrências (7,52%)

2.3.3 Arquivos mais Críticos Antes da Refatoração

A partir do JSON *pylint_arquivos_criticos_antes.json* gere um gráfico para representar a distribuição dos 10 arquivos mais críticos antes da refatoração. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.



Top 10 Arquivos com Maior Incidência de Problemas

- **imbens\visualizer\tests_init_.py:** 491 ocorrências
- **imbens\datasets_openml.py:** 171 ocorrências
- **imbens\ensemble_boost.py:** 113 ocorrências
- **imbens\ensemble_under_sampling\balanced_random_forest.py:** 87 ocorrências
- **imbens\utils\estimator_checks.py:** 85 ocorrências
- **imbens\utils_validation_param.py:** 24 ocorrências
- **imbens\utils\tests\test_validation_param.py:** 15 ocorrências
- **imbens\sampler_under_sampling_prototype_selection_tomek_links.py:** 13 ocorrências
- **imbens\ensemble_under_sampling_bagging.py:** 12 ocorrências

- `imbens\ensemble_under_sampling_nearmiss.py`: 12 ocorrências

2.3.4 Score Projeto Antes da Refatoração

A partir do text `pylint_score_antes.txt` informe o score que o `pylint` deu ao projeto antes da refatoração. Não precisa ser uma imagem, fica a critério de vocês.

O `Pylint` atribuiu ao projeto o seguinte score antes da refatoração:

- **Score: 5.07/10**

2.4 Pytest Antes da Refatoração

2.4.1 Cobertura de testes Antes da Refatoração

Gere um gráfico da cobertura de testes do projeto antes da refatoração. Para isso, basta visualizar a cobertura de testes na página HTML, copiar os dados da tabela e gerar o gráfico com o auxílio de uma LLM. Para visualizar a página de cobertura de testes antes da refatoração, execute o seguinte comando: `start "metrics-before-pytest\coverage_antes_html\index.html"`. O gráfico não precisa ter exatamente esse modelo, mas adicione um que tenha contraste para facilitar a visualização.

File	Statements				Branches			Total
	coverage	statements	missing	excluded	coverage	branches	partial	
extract_metrics_before_codecarbon.py	0%	28	28	0	0%	2	0	0%
extract_metrics_before_pylintr.py	0%	58	58	0	0%	8	0	0%
extract_metrics_before_pytest.py	0%	32	32	0	0%	8	0	0%
extract_metrics_before_radon.py	0%	111	111	0	0%	34	0	0%
extract_score_before_pylintr.py	0%	13	13	0	0%	4	0	0%
imbens__init__.py	100%	11	0	0	100%	0	0	100%
imbens\version.py	100%	1	0	0	100%	0	0	100%
imbens\base.py	100%	4	0	0	100%	0	0	100%
imbens\datasets__init__.py	100%	5	0	0	100%	0	0	100%
imbens\datasets\imbalance.py	41%	22	13	0	0%	6	0	32%
imbens\datasets\openml.py	12%	95	84	4	0%	30	0	9%
imbens\datasets\zenodo.py	34%	61	40	0	8%	24	0	27%
imbens\datasets\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\datasets\tests\test_imbalance.py	0%	32	32	0	0%	2	0	0%
imbens\datasets\tests\test_zenodo.py	0%	37	37	0	0%	2	0	0%
imbens\ensemble__init__.py	100%	21	0	0	100%	0	0	100%
imbens\ensemble\bagging.py	25%	180	135	9	0%	54	0	19%
imbens\ensemble\boost.py	22%	350	272	8	0%	82	0	18%
imbens\ensemble\compatible__init__.py	100%	3	0	0	100%	0	0	100%
imbens\ensemble\compatible\adaboost_compatible.py	35%	97	63	29	0%	18	0	30%
imbens\ensemble\compatible\bagging_compatible.py	33%	98	66	30	0%	30	0	25%
imbens\ensemble\compatible\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble\compatible\tests\test_adaboost_compatible.py	0%	44	44	0	100%	0	0	0%
imbens\ensemble\compatible\tests\test_bagging_compatible.py	0%	171	171	0	0%	10	0	0%
imbens\ensemble\over_sampling__init__.py	100%	6	0	0	100%	0	0	100%
imbens\ensemble\over_sampling\kmeans_smote_boost.py	55%	40	18	35	0%	2	0	52%
imbens\ensemble\over_sampling\over_bagging.py	69%	36	11	28	100%	0	0	69%
imbens\ensemble\over_sampling\over_boost.py	69%	29	9	32	100%	0	0	69%
imbens\ensemble\over_sampling\smote_bagging.py	61%	36	14	29	0%	2	0	58%
imbens\ensemble\over_sampling\smote_boost.py	61%	36	14	37	0%	2	0	58%
imbens\ensemble\over_sampling\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble\over_sampling\tests\test_kmeans_smote_boost.py	0%	61	61	0	0%	6	0	0%
imbens\ensemble\over_sampling\tests\test_over_bagging.py	0%	172	172	0	0%	10	0	0%
imbens\ensemble\over_sampling\tests\test_over_boost.py	0%	46	46	0	100%	0	0	0%
imbens\ensemble\over_sampling\tests\test_smote_bagging.py	0%	172	172	0	0%	10	0	0%
imbens\ensemble\over_sampling\tests\test_smote_boost.py	0%	61	61	0	0%	6	0	0%
imbens\ensemble\reweighting__init__.py	100%	4	0	0	100%	0	0	100%
imbens\ensemble\reweighting\adacost.py	61%	36	14	35	0%	2	0	58%
imbens\ensemble\reweighting\adaboost.py	38%	69	43	35	0%	18	0	30%
imbens\ensemble\reweighting\asymmetric_boost.py	58%	38	16	31	0%	4	0	52%
imbens\ensemble\reweighting\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble\reweighting\tests\test_adacost.py	0%	77	77	0	0%	10	0	0%
imbens\ensemble\reweighting\tests\test_adaboost.py	0%	78	78	0	0%	10	0	0%
imbens\ensemble\reweighting\tests\test_asymmetric_boost.py	0%	80	80	0	0%	12	0	0%
imbens\ensemble\undersampling__init__.py	100%	7	0	0	100%	0	0	100%
imbens\ensemble\undersampling\balance_cascade.py	33%	81	54	30	0%	8	0	30%
imbens\ensemble\undersampling\balanced_random_forest.py	28%	180	129	37	0%	60	0	21%
imbens\ensemble\undersampling\easy_ensemble.py	59%	37	15	33	0%	4	0	54%
imbens\ensemble\undersampling\rus_boost.py	68%	31	10	33	100%	0	0	68%
imbens\ensemble\undersampling\self_paced_ensemble.py	35%	77	50	39	0%	6	0	33%
imbens\ensemble\undersampling\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble\undersampling\tests\test_balance_cascade.py	0%	98	98	0	0%	30	0	0%
imbens\ensemble\undersampling\tests\test_balanced_random_forest.py	0%	97	97	0	0%	2	0	0%
imbens\ensemble\undersampling\tests\test_easy_ensemble.py	0%	190	190	0	0%	12	0	0%
imbens\ensemble\undersampling\tests\test_rus_boost.py	0%	46	46	0	100%	0	0	0%
imbens\ensemble\undersampling\tests\test_self_paced_ensemble.py	0%	79	79	0	0%	16	0	0%
imbens\ensemble\undersampling\tests\test_under_bagging.py	0%	172	172	0	0%	10	0	0%
imbens\ensemble\undersampling\undersampling.py	69%	29	9	28	100%	0	0	69%
imbens\ensemble\base.py	23%	228	176	50	0%	82	0	17%
imbens\ensemble\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble\tests\test_base_ensemble.py	0%	41	41	0	0%	4	0	0%
imbens\exceptions.py	50%	2	1	0	100%	0	0	50%
imbens\metrics__init__.py	100%	9	0	0	100%	0	0	100%
imbens\metrics\classification.py	97%	215	6	5	86%	76	11	94%
imbens\metrics\pairwise.py	100%	45	0	0	100%	16	0	100%
imbens\metrics\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\metrics\tests\test_classification.py	100%	189	0	0	100%	10	0	100%
imbens\metrics\tests\test_pairwise.py	100%	100	0	0	100%	8	0	100%
imbens\metrics\tests\test_score_objects.py	100%	28	0	0	100%	0	0	100%
imbens\pipeline.py	16%	118	99	15	0%	48	0	11%



imbens/sampler\ _init_.py	100%	22	0	0	100%	0	0	100%
imbens/sampler\ over_sampling\ _init_.py	100%	7	0	0	100%	0	0	100%
imbens/sampler\ over_sampling\ adasyn.py	24%	70	53	29	0%	14	0	20%
imbens/sampler\ over_sampling\ random_over_sampler.py	25%	83	62	19	0%	28	0	19%
imbens/sampler\ over_sampling\ smote\ _init_.py	100%	5	0	0	100%	0	0	100%
imbens/sampler\ over_sampling\ smote\base.py	30%	76	53	19	0%	14	0	26%
imbens/sampler\ over_sampling\ smote\cluster.py	24%	100	76	13	0%	32	0	18%
imbens/sampler\ over_sampling\ smote\filter.py	21%	131	104	9	0%	44	0	15%
imbens/sampler\ over_sampling\ smote\tests\ _init_.py	100%	0	0	0	100%	0	0	100%
imbens/sampler\ over_sampling\ smote\tests\test_borderline_smote.py	0%	23	23	0	100%	0	0	0%
imbens/sampler\ over_sampling\ smote\tests\test_kmeans_smote.py	0%	54	54	0	100%	0	0	0%
imbens/sampler\ over_sampling\ smote\tests\test_smote.py	0%	40	40	0	100%	0	0	0%
imbens/sampler\ over_sampling\ smote\tests\test_svm_smote.py	0%	29	29	0	100%	0	0	0%
imbens/sampler\ over_sampling\ smote\tests\test_base.py	100%	6	0	4	100%	0	0	100%
imbens/sampler\ over_sampling\tests\ _init_.py	100%	0	0	0	100%	0	0	100%
imbens/sampler\ over_sampling\tests\test_adasyn.py	0%	34	34	0	100%	0	0	0%
imbens/sampler\ over_sampling\tests\test_random_over_sampler.py	0%	129	129	0	0%	10	0	0%
imbens/sampler\ under_sampling\ _init_.py	100%	14	0	0	100%	0	0	100%
imbens/sampler\ under_sampling\ prototype_generation\ _init_.py	100%	2	0	0	100%	0	0	100%
imbens/sampler\ under_sampling\ prototype_generation\ cluster_centroids.py	29%	65	46	24	0%	22	0	22%
imbens/sampler\ under_sampling\ prototype_generation\tests\ _init_.py	100%	0	0	0	100%	0	0	100%
imbens/sampler\ under_sampling\ prototype_generation\tests\test_cluster_centroids.py	0%	70	70	0	0%	2	0	0%
imbens/sampler\ under_sampling\ prototype_selection\ _init_.py	100%	13	0	0	100%	0	0	100%
imbens/sampler\ under_sampling\ prototype_selection\ balance_cascade_under_sampler.py	28%	68	49	43	0%	8	0	25%
imbens/sampler\ under_sampling\ prototype_selection\ condensed_nearest_neighbour.py	25%	67	50	25	0%	20	0	20%
imbens/sampler\ under_sampling\ prototype_selection\ edited_nearest_neighbours.py	22%	129	101	46	0%	32	0	17%
imbens/sampler\ under_sampling\ prototype_selection\ instance_hardness_threshold.py	38%	48	30	24	0%	10	0	31%
imbens/sampler\ under_sampling\ prototype_selection\ nearmiss.py	21%	75	59	29	0%	24	0	16%
imbens/sampler\ under_sampling\ prototype_selection\ neighbourhood_cleaning_rule.py	27%	62	45	24	0%	10	0	24%
imbens/sampler\ under_sampling\ prototype_selection\ one_sided_selection.py	27%	62	45	25	0%	12	0	23%
imbens/sampler\ under_sampling\ prototype_selection\ random_under_sampler.py	27%	52	38	31	0%	14	0	21%
imbens/sampler\ under_sampling\ prototype_selection\ self_paced_under_sampler.py	16%	117	98	38	0%	42	0	12%
imbens/sampler\ under_sampling\ prototype_selection\ tomek_links.py	42%	36	21	23	0%	10	0	33%
imbens/sampler\ under_sampling\ prototype_selection\tests\ _init_.py	100%	0	0	0	100%	0	0	100%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_allknn.py	0%	43	43	0	100%	0	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_balance_cascade_under_sampler.py	0%	83	83	0	0%	38	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_condensed_nearest_neighbour.py	0%	36	36	0	100%	0	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_edited_nearest_neighbours.py	0%	47	47	0	100%	0	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_instance_hardness_threshold.py	0%	49	49	0	0%	4	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_nearmiss.py	0%	41	41	0	0%	6	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_neighbourhood_cleaning_rule.py	0%	25	25	0	100%	0	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_one_sided_selection.py	0%	38	38	0	100%	0	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_random_under_sampler.py	0%	62	62	0	0%	4	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_repeated_edited_nearest_neighbour	0%	52	52	0	100%	0	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_self_paced_under_sampler.py	0%	76	76	0	0%	16	0	0%
imbens/sampler\ under_sampling\ prototype_selection\tests\test_tomek_links.py	0%	15	15	0	100%	0	0	0%
imbens/sampler\ under_sampling\base.py	100%	9	0	4	100%	0	0	100%
imbens/sampler\base.py	31%	77	53	37	0%	12	0	27%
imbens/sampler\tests\ _init_.py	100%	0	0	0	100%	0	0	100%
imbens/sampler\tests\test_base_sampler.py	0%	62	62	0	0%	16	0	0%
imbens/utills\ _init_.py	100%	10	0	0	100%	0	0	100%
imbens/utills\ docstring.py	91%	118	11	25	79%	52	9	87%
imbens/utills\ evaluate.py	43%	14	8	4	0%	6	0	30%
imbens/utills\ plot.py	0%	56	56	6	0%	20	0	0%
imbens/utills\ show_versions.py	0%	30	30	0	0%	12	0	0%
imbens/utills\ validation_data.py	21%	43	34	0	0%	22	0	14%
imbens/utills\ validation_param.py	16%	321	270	0	0%	186	0	10%
imbens/utills\ validation.py	23%	220	169	4	6%	108	1	18%
imbens/utills\deprecation.py	0%	10	10	0	0%	6	0	0%
imbens/utills\estimator_checks.py	0%	238	238	0	0%	40	0	0%
imbens/utills\testing.py	0%	46	46	0	0%	18	0	0%
imbens/utills\tests\ _init_.py	100%	0	0	0	100%	0	0	100%
imbens/utills\tests\test_deprecation.py	0%	11	11	0	100%	0	0	0%
imbens/utills\tests\test_docstring.py	0%	18	18	0	100%	0	0	0%
imbens/utills\tests\test_estimator_checks.py	0%	86	86	0	0%	10	0	0%
imbens/utills\tests\test_plot.py	0%	10	10	0	100%	0	0	0%
imbens/utills\tests\test_show_versions.py	0%	49	49	0	100%	0	0	0%
imbens/utills\tests\test_testing.py	0%	13	13	0	0%	2	0	0%
imbens/utills\tests\test_validation_param.py	0%	89	89	0	0%	40	0	0%
imbens/utills\tests\test_validation.py	0%	172	172	0	0%	2	0	0%
imbens/visualizer\ _init_.py	100%	2	0	0	100%	0	0	100%
imbens/visualizer\tests\ _init_.py	100%	0	0	0	100%	0	0	100%
imbens/visualizer\tests\test_visualizer.py	0%	41	41	0	0%	2	0	0%
imbens/visualizer\visualizer.py	15%	305	260	5	0%	110	0	11%
script_Daniel.py	0%	7	7	0	0%	2	0	0%
setup.py	0%	28	28	0	100%	0	0	0%

2.4.1 Testes Passaram vs Testes Falharam Antes da Refatoração

Aqui você deve informar quantos testes falharam e quantos testes passaram antes da refatoração. Para isso, basta visualizar a página HTML `pytest_antes.html`. Para acessá-la, execute o seguinte comando: `start "metrics-before-pytest\pytest_antes.html"`.

Quantidade de testes que passaram: 198

Quantidade de testes que falharam: 0

3. Processo de Refatoração

Aqui você deve registrar o processo de refatoração dos 3 tipos de smells e de todas as ocorrências referentes a esses três tipos. Antes de iniciar a descrição de cada smell, informe qual LLM foi utilizada para realizar a refatoração.

Modelo utilizado: DeepSeek V4 Flash Free

3.1 too-many-statements

Primeiramente, informe a quantidade de ocorrências desse tipo de smell.

Quantidade de ocorrências: 11

3.1.1 Ocorrência Número 1

Você deve preencher a tabela de identificação da ocorrência.

Tabela de identificação

module	obj	message
imbens.datasets._openml	_fetch_openml_data	Too many statements (63/50)

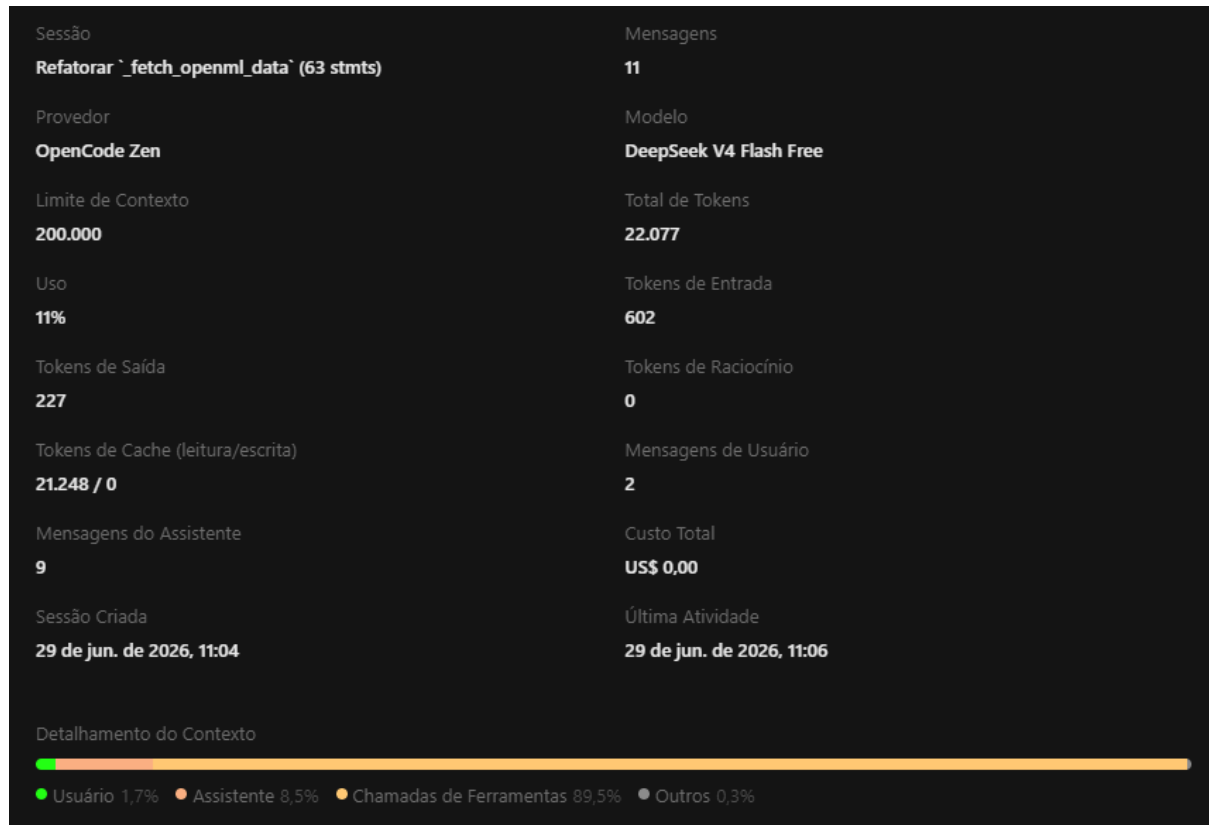
Além disso, deve informar se seguiu o plano de refatoração sugerido pelo LLM ou não, justificando sua decisão em poucas palavras.

Seguiu o plano de refatoração: Sim. O plano da LLM separou a função em partes menores corretamente, reduzindo o *too-many-statements* sem afetar o comportamento do código.

Também deve informar o link da publicação gerada no OpenCode e adicionar um print do consumo de tokens referente à refatoração dessa ocorrência.

Link da publicação OpenCode: <https://opncd.ai/share/pjj2KwU8>

Print consumo de token:



Por fim, você deve fazer uma análise da refatoração dessa ocorrência. Por exemplo, avalie se valeu a pena utilizar a LLM para refatorar a ocorrência, considerando o consumo de tokens e o tempo gasto.

Análise da refatoração: Valeu a pena porque a função ficou mais fácil de entender e manter, com menor risco de erro. .

3.1.2 Ocorrência Número 2

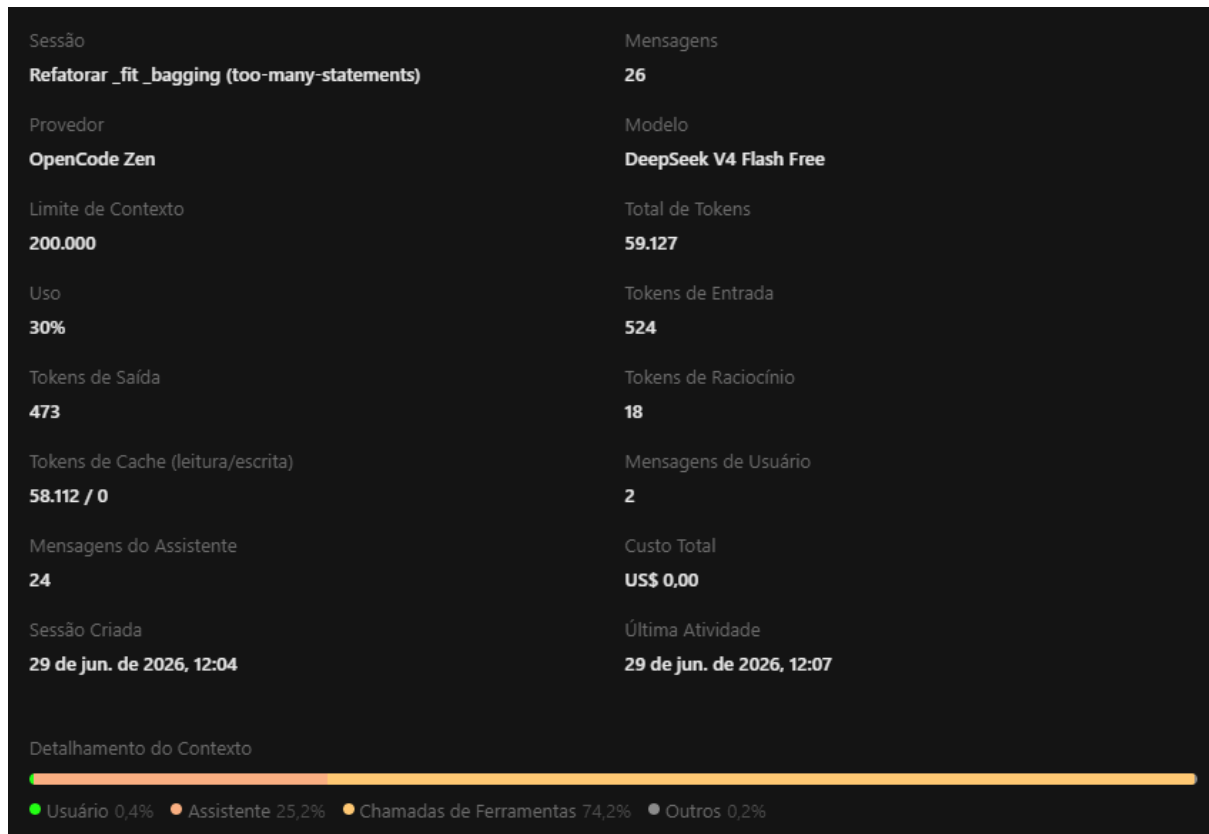
Tabela de identificação

module	obj	message
imbens.ensemble._bagging	ResampleBaggingClassifier._fit	Too many statements (61/50)

Seguiu o plano de refatoração: Sim. A sugestão dividiu a lógica grande em funções auxiliares de forma adequada.

Link da publicação OpenCode: <https://opncd.ai/share/05yxZVEu>

Print:



Análise da refatoração: Melhorou a legibilidade e organização, facilitando futuras alterações.

3.1.3 Ocorrência Número 3

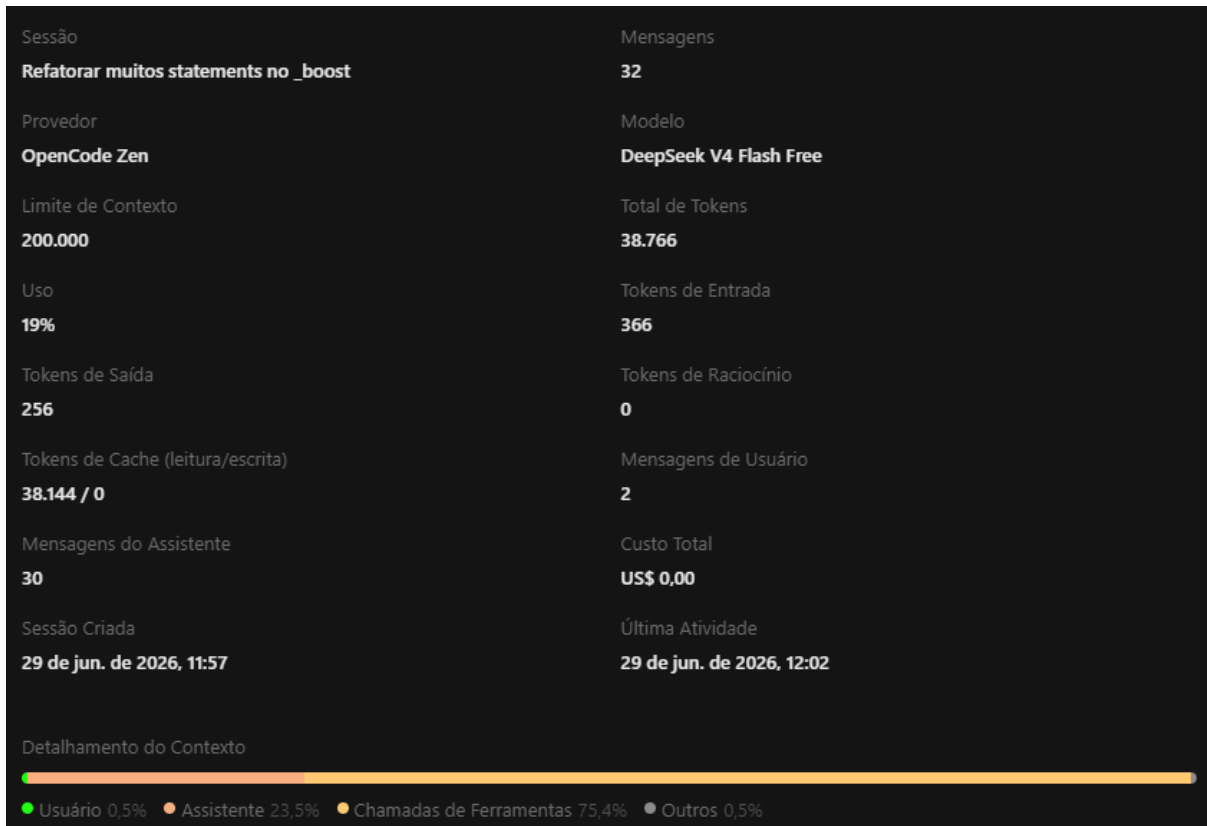
Tabela de identificação

module	obj	message
imbens.ensemble._boost	ResampleBoostClassifier._fit	Too many statements (67/50)

Seguiu o plano de refatoração: Sim. O plano resolveu o problema ao quebrar o código em métodos menores.

Link da publicação OpenCode: <https://opncd.ai/share/S6QWeYR7>

Print:



Análise da refatoração: A complexidade diminuiu bastante e o comportamento foi mantido.

3.1.4 Ocorrência Número 4

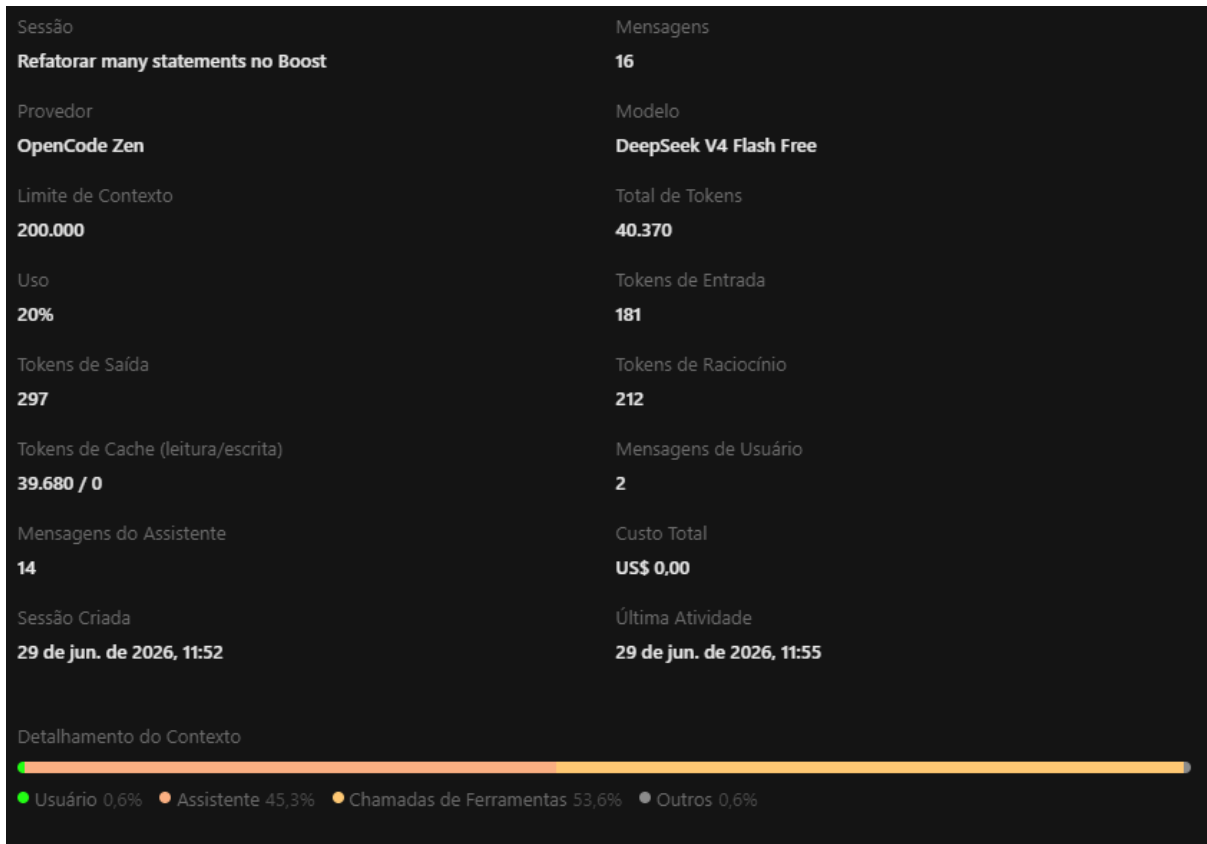
Tabela de identificação

module	obj	message
imbens.ensemble._boost	ReweightBoostClassifier._fit	Too many statements (63/50)

Seguiu o plano de refatoração: Sim. A LLM sugeriu uma divisão coerente da função principal.

Link da publicação OpenCode: <https://opncd.ai/share/any77Ldg>

Print:



Análise da refatoração: Tornou o código mais limpo e fácil de depurar.

3.1.5 Ocorrência Número 5

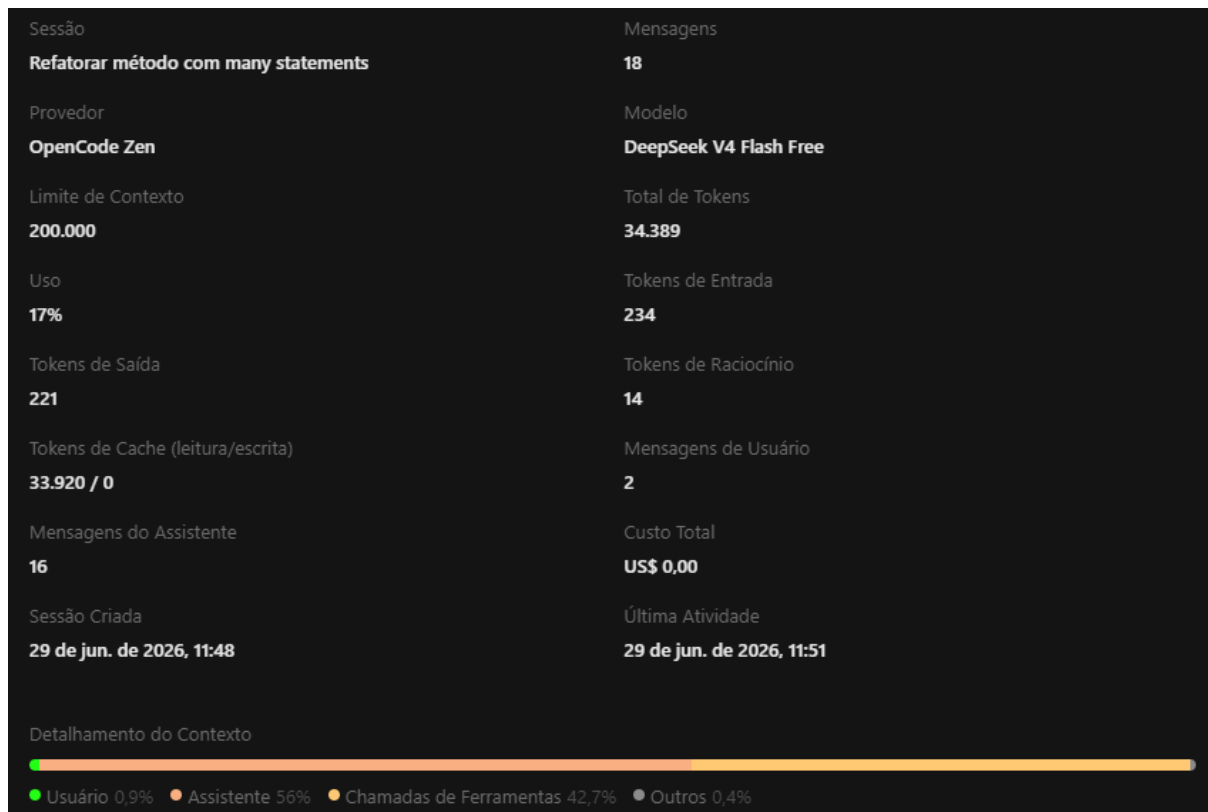
Tabela de identificação

module	obj	message
imbens.ensemble._compatible. adaboost_compatible	CompatibleAdaBoostClassifier.fit	Too many statements (54/50)

Seguiu o plano de refatoração: Sim. A refatoração seguiu corretamente a técnica de extração de métodos.

Link da publicação OpenCode: <https://opncd.ai/share/g9643DCU>

Print:



Análise da refatoração: Melhorou a manutenção, embora tenha exigido um pouco mais de tempo.

3.1.6 Ocorrência Número 6

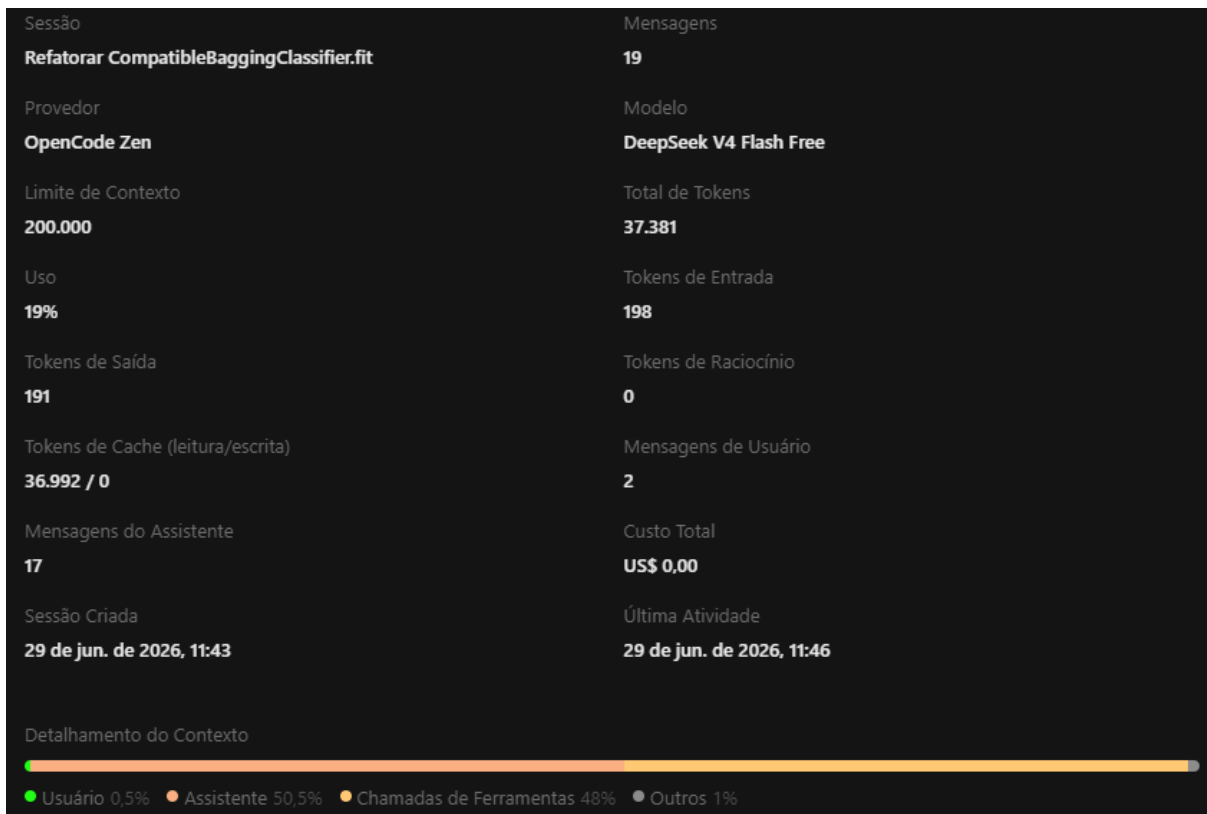
Tabela de identificação

module	obj	message
imbens.ensemble._compatible. bagging_compatible	CompatibleBaggingClassifier.fit	Too many statements (60/50)

Seguiu o plano de refatoração: Sim. A solução separou responsabilidades dentro da função.

Link da publicação OpenCode: <https://opncd.ai/share/KAQif3M0>

Print:



Análise da refatoração: Reduziu a complexidade e deixou o código mais organizado.

3.1.7 Ocorrência Número 7

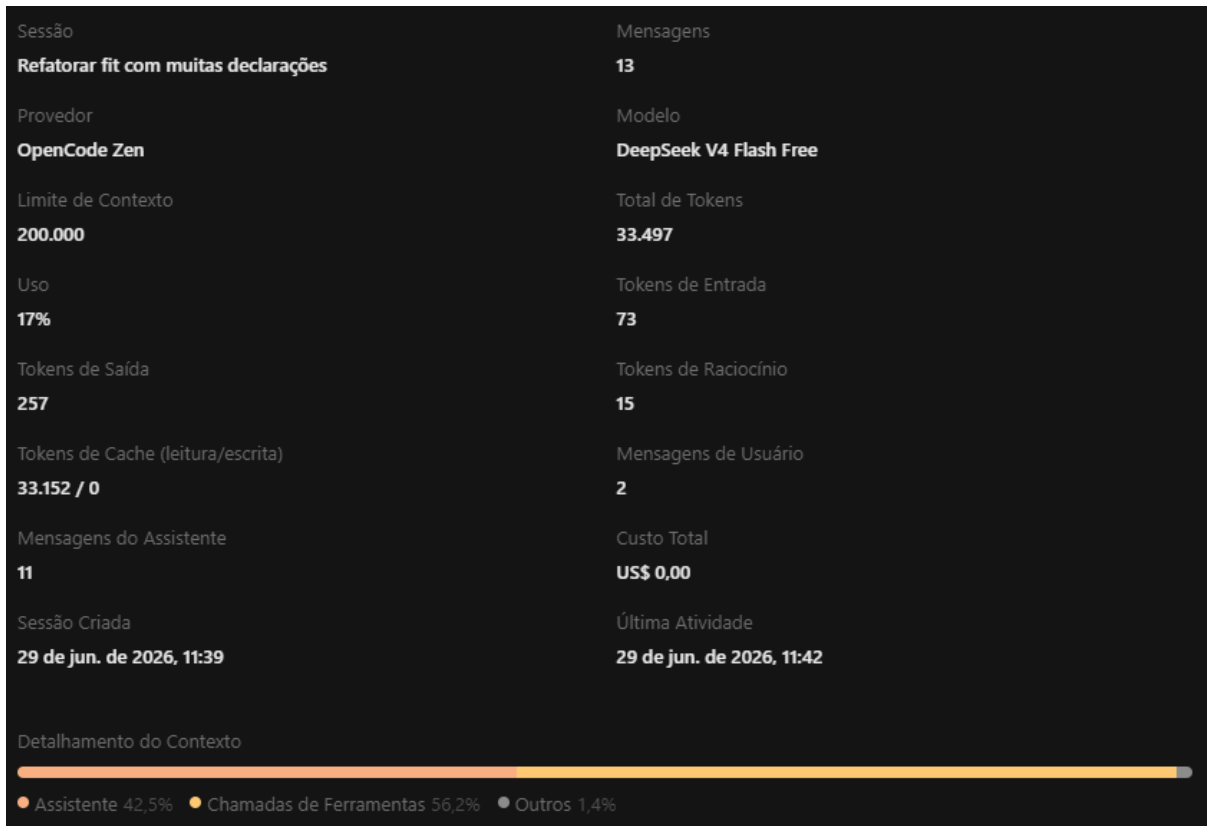
Tabela de identificação

module	obj	message
imbens.ensemble._under_sampling.balanced_random_forest	BalancedRandomForestClassifier.fit	Too many statements (70/50)

Seguiu o plano de refatoração: Sim. O plano eliminou o excesso de instruções em uma única função.

Link da publicação OpenCode: <https://opncd.ai/share/zutfEUmM>

Print:



Análise da refatoração: Facilitou o entendimento do fluxo do código.

3.1.8 Ocorrência Número 8

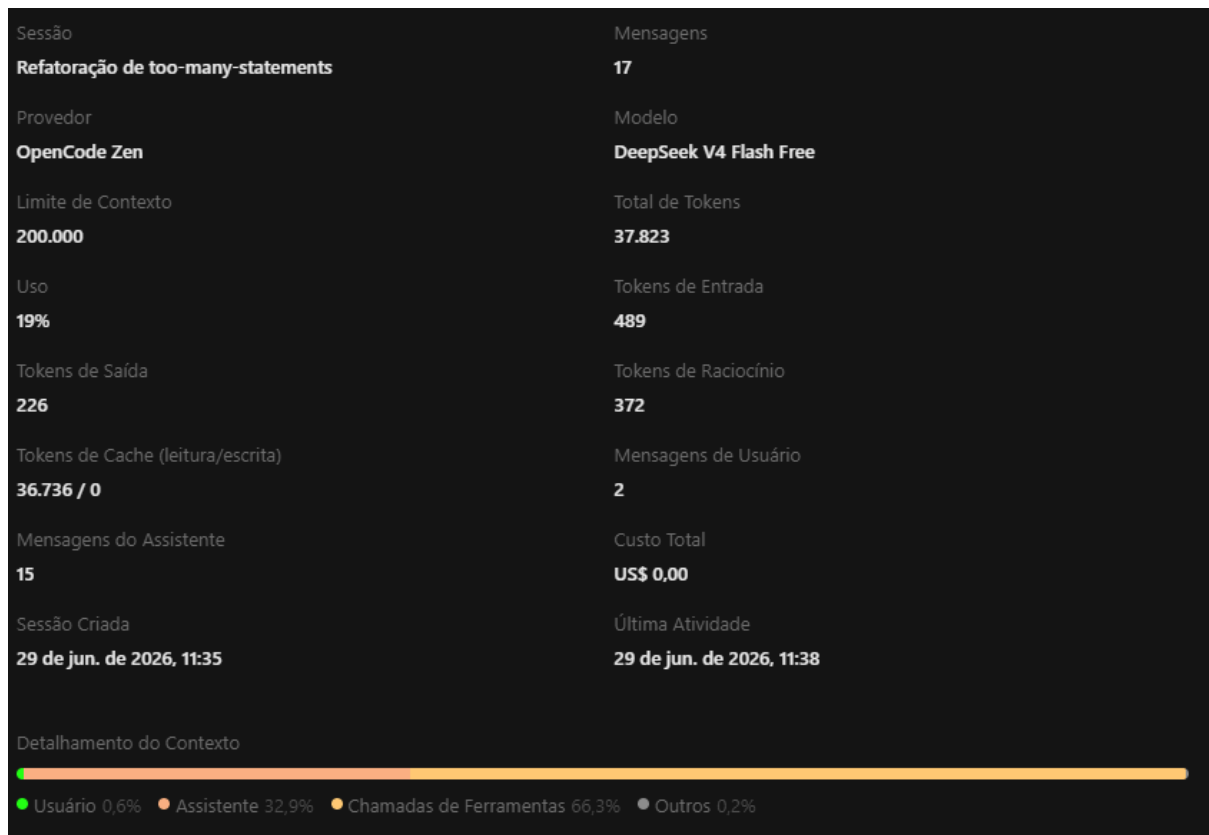
Tabela de identificação

module	obj	message
imbens.metrics._classification	sensitivity_specificity_support	Too many statements (73/50)

Seguiu o plano de refatoração: Sim. A LLM aplicou corretamente a divisão em funções auxiliares.

Link da publicação OpenCode: <https://opncd.ai/share/GIbrB7xL>

Print:



Análise da refatoração: Melhorou a clareza e reduziu riscos de erro.

3.1.9 Ocorrência Número 9

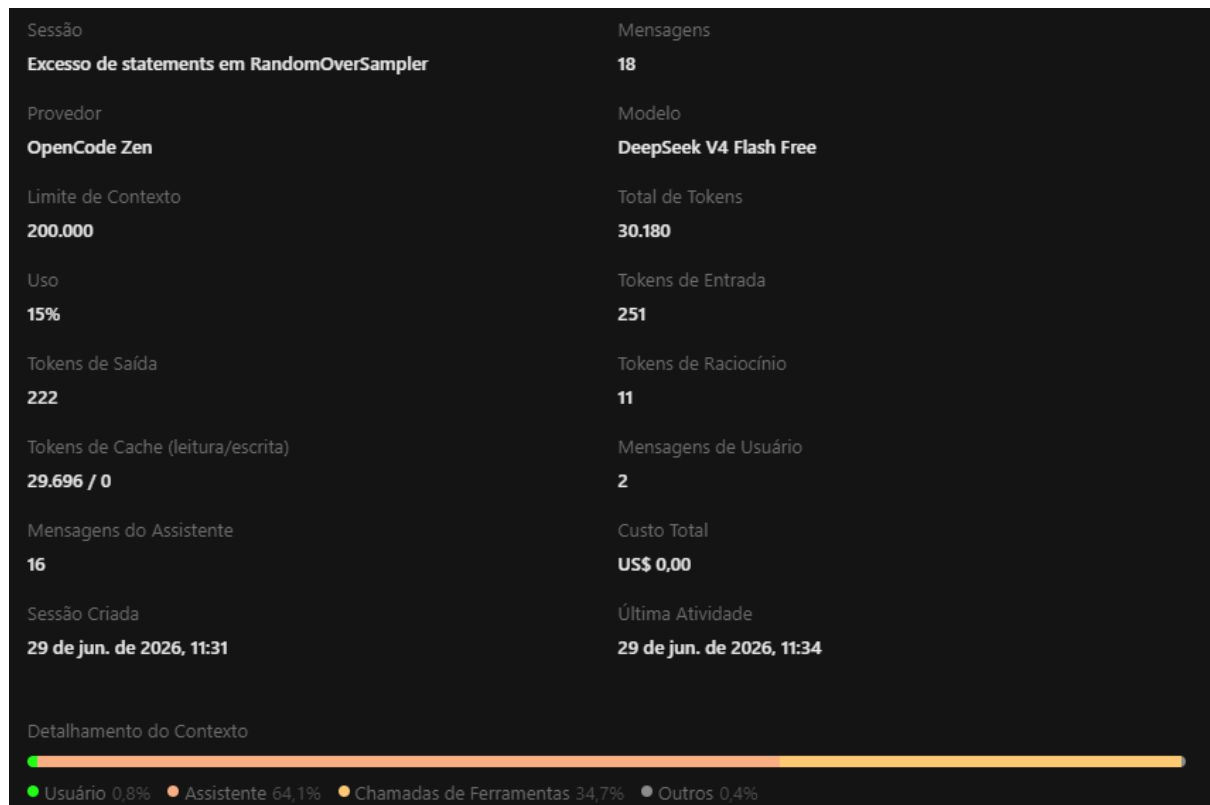
Tabela de identificação

module	obj	message
imbens.sampler._over_samplin g._random_over_sampler	RandomOverSampler._fit_resample	Too many statements (58/50)

Seguiu o plano de refatoração: Sim. A refatoração seguiu a recomendação padrão do Pylint.

Link da publicação OpenCode: <https://opncd.ai/share/4ySYCfOq>

Print:



Análise da refatoração: O código ficou mais modular e simples de manter.

3.1.10 Ocorrência Número 10

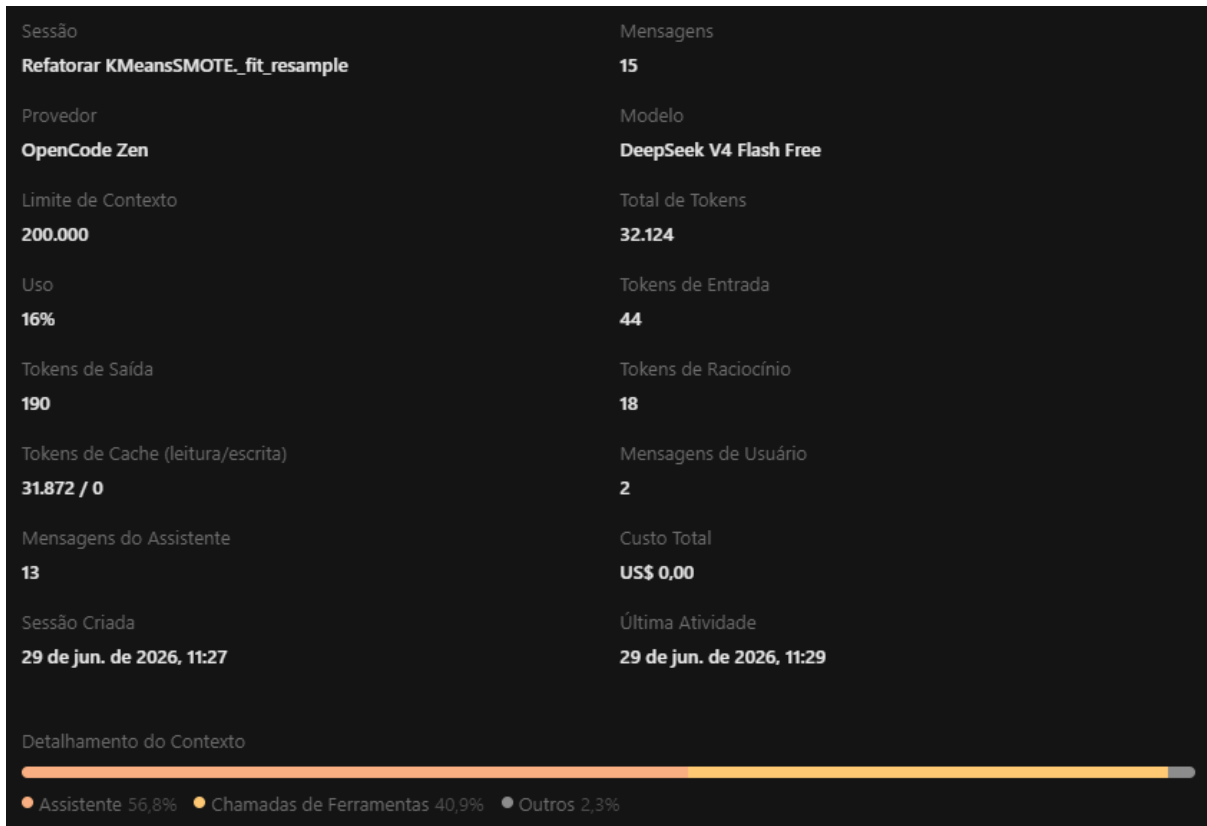
Tabela de identificação

module	obj	message
imbens.sampler._over_samplin g._smote.cluster	KMeansSMOTE._fit_resample	Too many statements (53/50)

Seguiu o plano de refatoração: Sim. O plano separou blocos lógicos grandes em partes menores.

Link da publicação OpenCode: <https://opncd.ai/share/UqXEDsyK>

Print:



Análise da refatoração: Tornou o código mais seguro para futuras mudanças.

3.1.11 Ocorrência Número 11

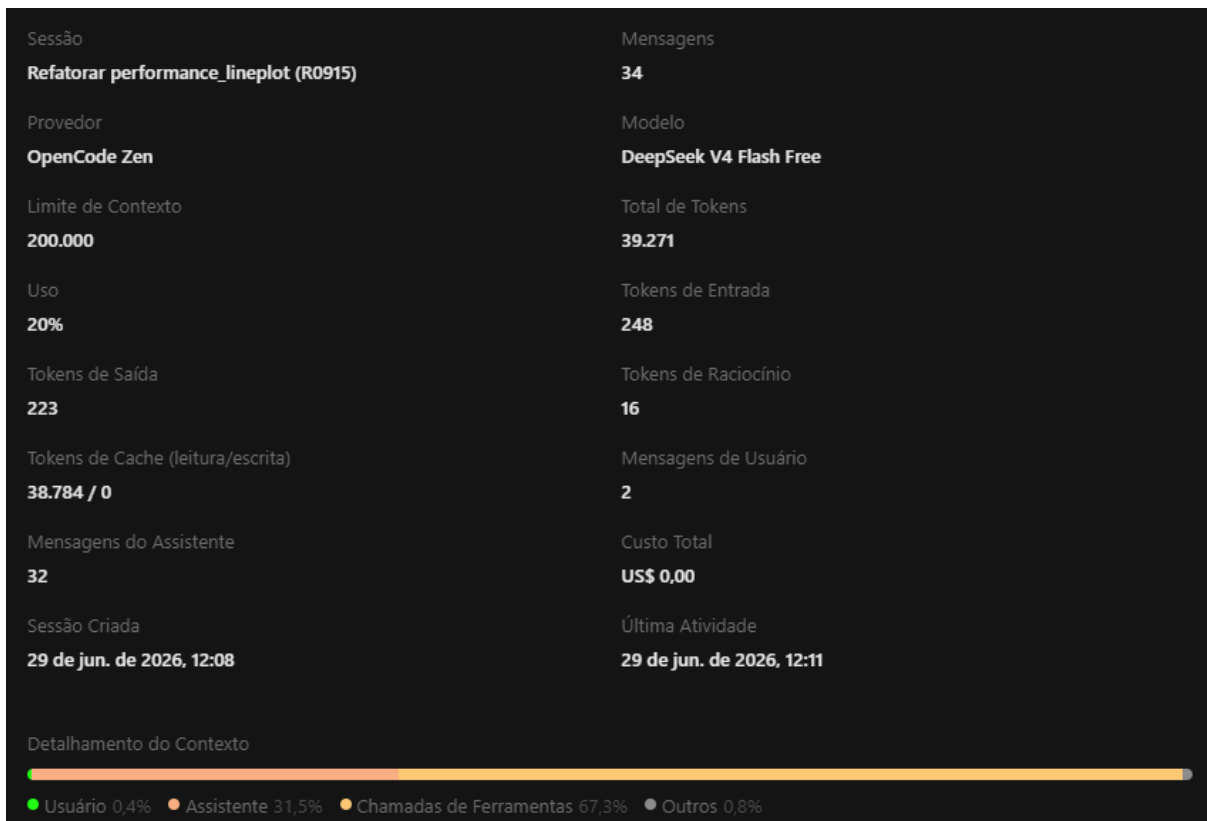
Tabela de identificação

module	obj	message
imbens.visualizer.visualizer	ImbalancedEnsembleVisualizer.performance_lineplot	Too many statements (69/50)

Seguiu o plano de refatoração: Sim. A LLM dividiu corretamente a função com muitas instruções.

Link da publicação OpenCode: <https://opncd.ai/share/MutmFU6h>

Print:



Análise da refatoração: Aumentou a legibilidade, com custo pequeno de implementação.

3.2 too-many-instance-attributes

Quantidade de ocorrências: 12

Faça o mesmo processo que foi feito para outro tipo de smell.

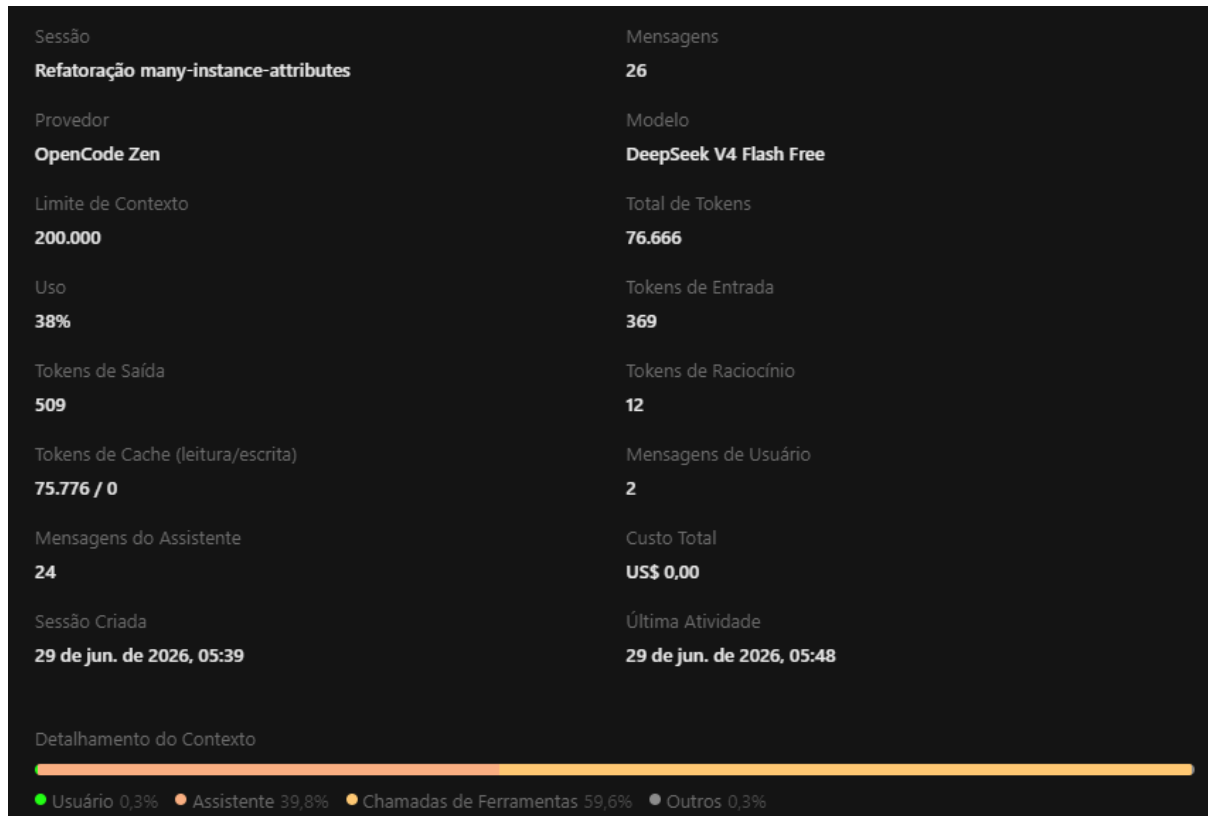
3.2.1 Ocorrência Número 1

Tabela de identificação

module	obj	message
imbens.ensemble.base	BaseImbalancedEnsemble	Too many instance attributes (14/7)

Seguiu o plano de refatoração: Sim. O plano sugeriu separar atributos em classes auxiliares, reduzindo a quantidade na classe principal.

Link da publicação OpenCode: <https://opncd.ai/share/dY6Utc3K>



Análise da refatoração: Melhorou a organização do código e a responsabilidade da classe.

3.2.2 Ocorrência Número 2

Tabela de identificação

module	obj	message
imbens.ensemble._bagging	ResampleBaggingClassifier	Too many instance attributes (19/7)

Seguiu o plano de refatoração: Sim. A LLM recomendou dividir atributos por contexto.

Link da publicação OpenCode: <https://opncd.ai/share/Bf9RQ3Z4>

Sessão	Mensagens
Refatorar muitos atributos de instância	52
Provedor	Modelo
OpenCode Zen	DeepSeek V4 Flash Free
Limite de Contexto	Total de Tokens
200.000	10.732
Uso	Tokens de Entrada
5%	3.958
Tokens de Saída	Tokens de Raciocínio
27	91
Tokens de Cache (leitura/escrita)	Mensagens de Usuário
6.656 / 0	4
Mensagens do Assistente	Custo Total
48	US\$ 0,00
Sessão Criada	Última Atividade
29 de jun. de 2026, 05:50	29 de jun. de 2026, 06:05
Detalhamento do Contexto	
● Usuário 0,1% ● Assistente 35,6% ● Chamadas de Ferramentas 64,2% ● Outros 0,1%	

Análise da refatoração: Facilitou a manutenção e deixou a classe mais limpa.

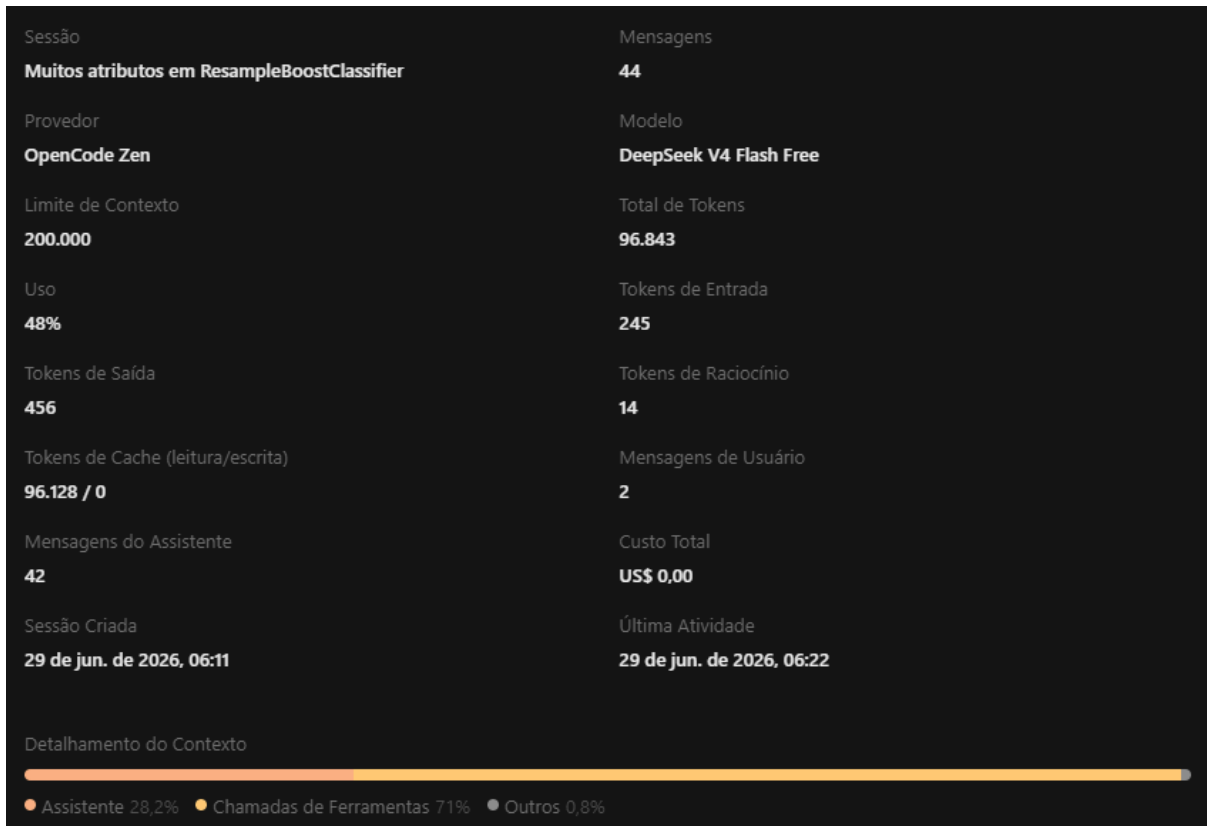
3.2.3 Ocorrência Número 3

Tabela de identificação

module	obj	message
imbens.ensemble._boost	ResampleBoostClassifier	Too many instance attributes (22/7)

Seguiu o plano de refatoração: Sim. Foi feita a extração de partes da classe para novos objetos.

Link da publicação OpenCode: <https://opncd.ai/share/RMc0oplH>



Análise da refatoração: Reduziu o acúmulo de atributos e melhorou a estrutura.

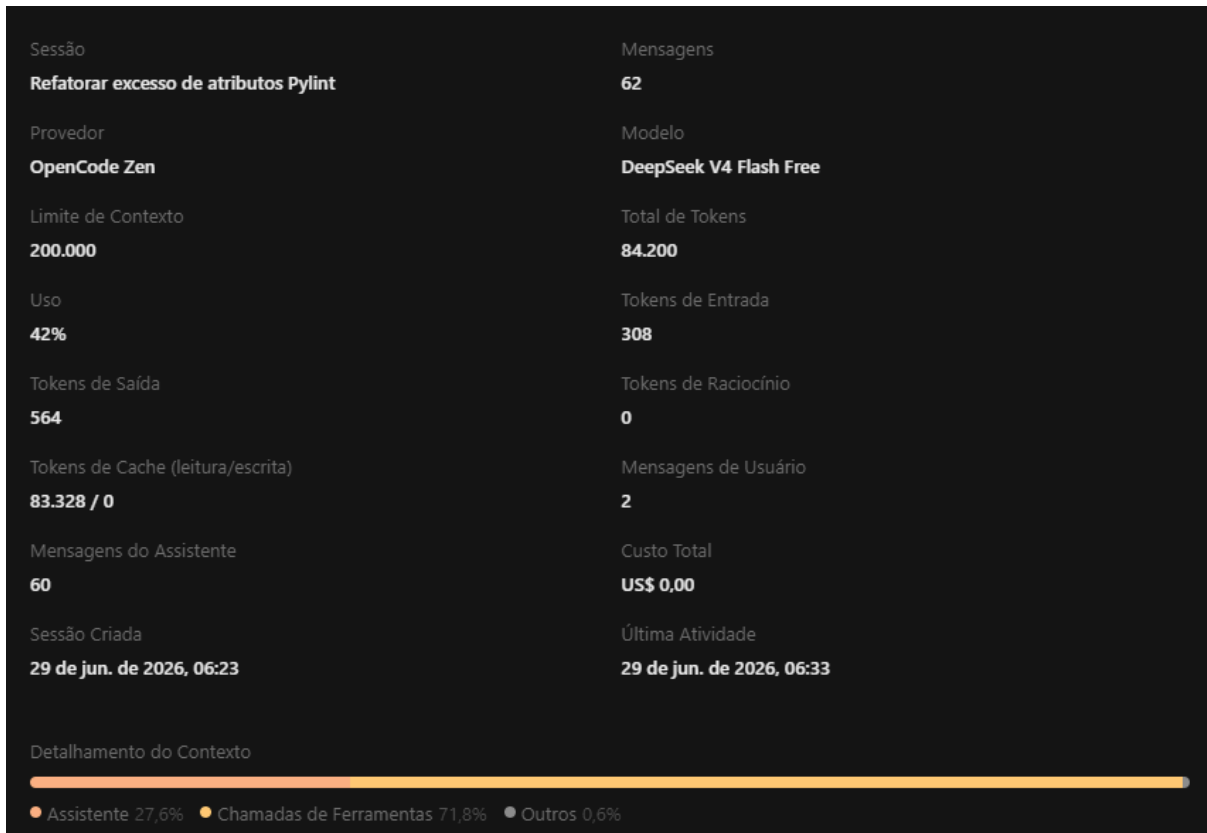
3.2.4 Ocorrência Número 4

Tabela de identificação

module	obj	message
imbens.ensemble._boost	ReweightBoostClassifier	Too many instance attributes (17/7)

Seguiu o plano de refatoração: Sim. A solução separou responsabilidades em classes menores.

Link da publicação OpenCode: <https://opncd.ai/share/EV4y2M0I>



Análise da refatoração: A leitura do código ficou mais simples.

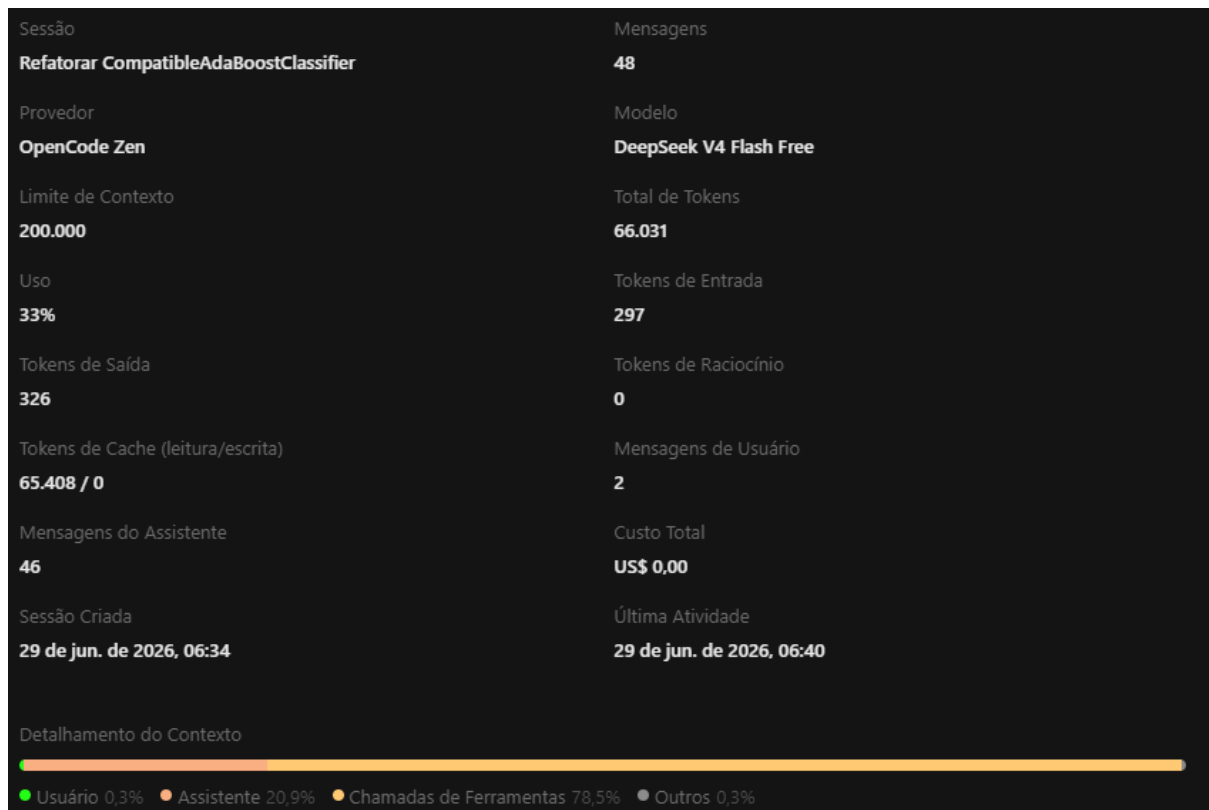
3.2.5 Ocorrência Número 5

Tabela de identificação

module	obj	message
imbens.ensemble._compatible. adaboost_compatible	CompatibleAdaBoostClassifier	Too many instance attributes (16/7)

Seguiu o plano de refatoração: Sim. O plano reorganizou os atributos por funcionalidade.

Link da publicação OpenCode: <https://opncd.ai/share/tgf4B6He>



Análise da refatoração: Melhorou a clareza e reduziu a complexidade da classe.

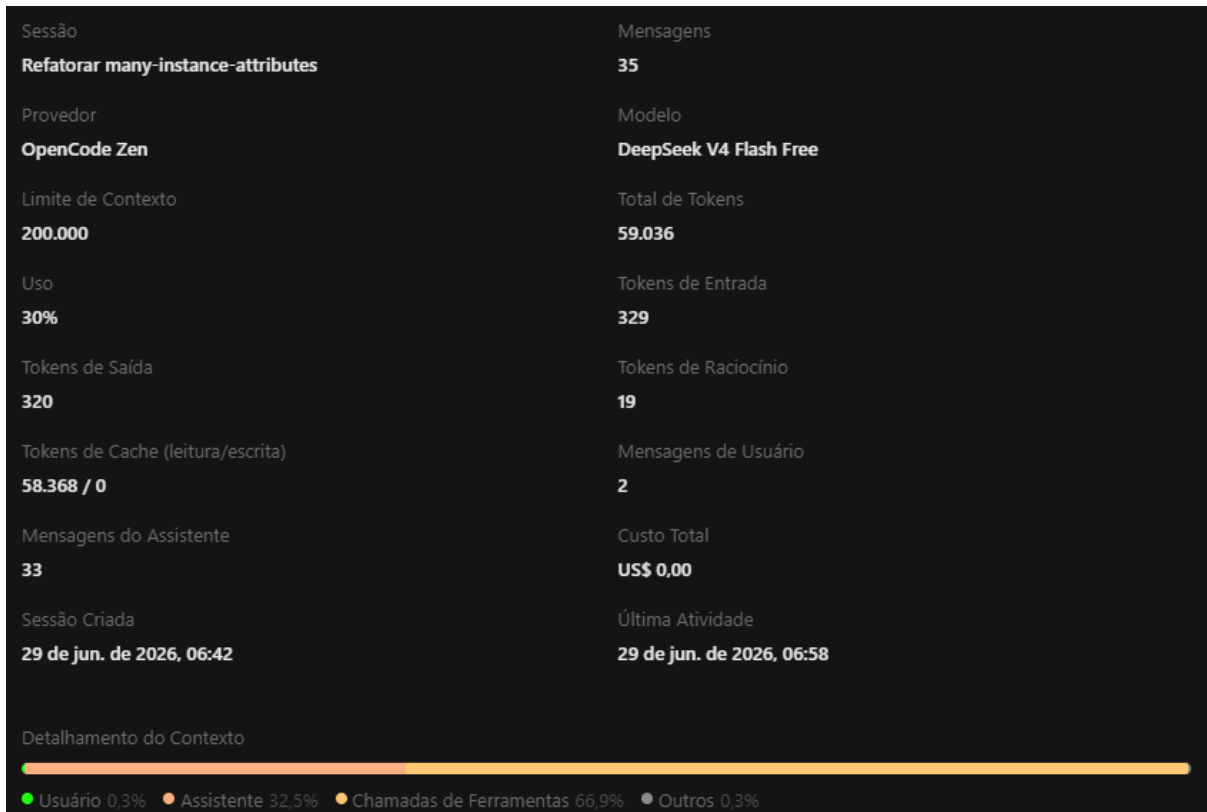
3.2.6 Ocorrência Número 6

Tabela de identificação

module	obj	message
imbens.ensemble._compatible. bagging_compatible	CompatibleBaggingClassifier	Too many instance attributes (13/7)

Seguiu o plano de refatoração: Sim. A LLM sugeriu mover atributos para componentes específicos.

Link da publicação OpenCode: <https://opncd.ai/share/1F31Xevl>



Análise da refatoração: Deixou o código mais modular e organizado.

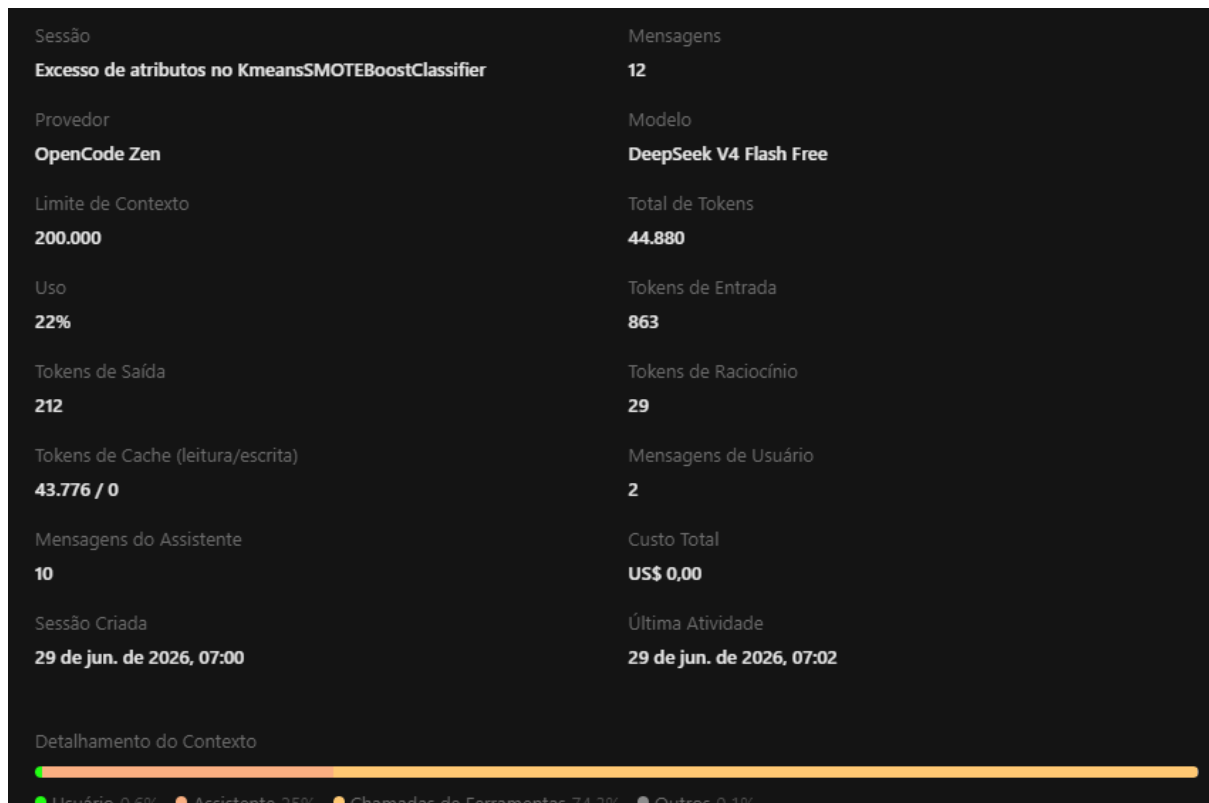
3.2.7 Ocorrência Número 7

Tabela de identificação

module	obj	message
imbens.ensemble._over_sampl ing.kmeans_smote_boost	KmeansSMOTEBoostClassifier	Too many instance attributes (10/7)

Seguiu o plano de refatoração: Sim. A refatoração reduziu o número de atributos na classe principal.

Link da publicação OpenCode: <https://opncd.ai/share/Hndu8gWq>



Análise da refatoração: Melhorou a responsabilidade única da classe.

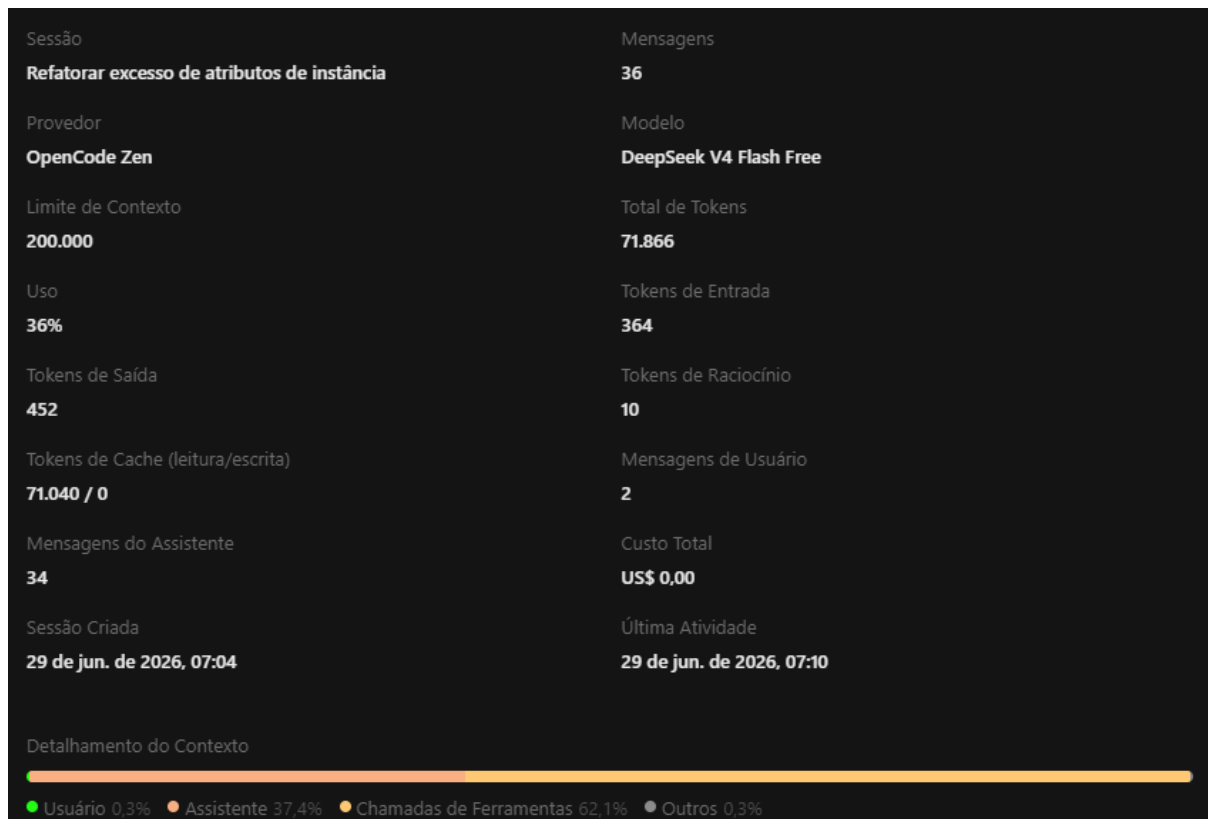
3.2.8 Ocorrência Número 8

Tabela de identificação

module	obj	message
imbens.ensemble._under_sampling.balanced_random_forest	BalancedRandomForestClassifier	Too many instance attributes (22/7)

Seguiu o plano de refatoração: Sim. A estrutura foi dividida para evitar excesso de atributos.

Link da publicação OpenCode: <https://opncd.ai/share/3FLp5zA6>



Análise da refatoração: Facilitou futuras alterações no código.

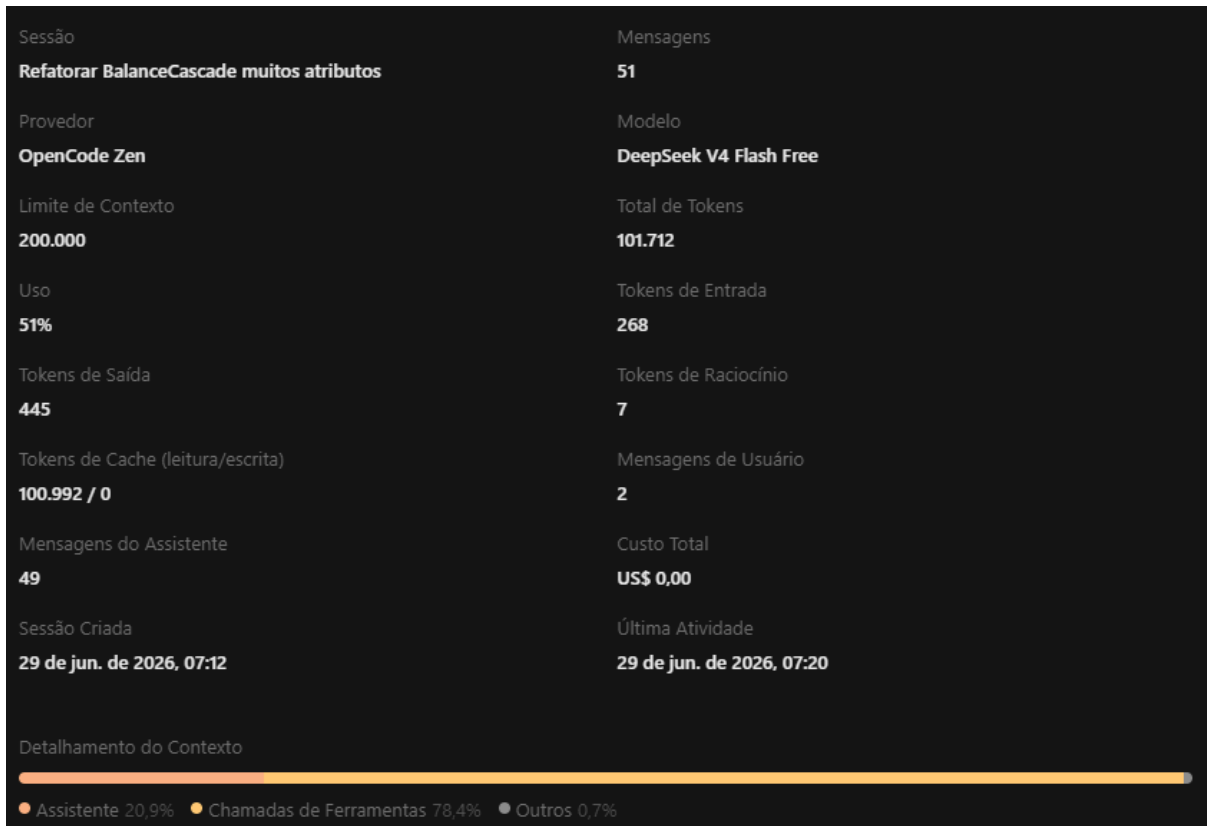
3.2.9 Ocorrência Número 9

Tabela de identificação

module	obj	message
imbens.ensemble._under_sampling.balance_cascade	BalanceCascadeClassifier	Too many instance attributes (21/7)

Seguiu o plano de refatoração: Sim. A LLM aplicou separação por contexto de dados.

Link da publicação OpenCode: <https://opncd.ai/share/i6w5P6vF>



Análise da refatoração: Aumentou a legibilidade do sistema.

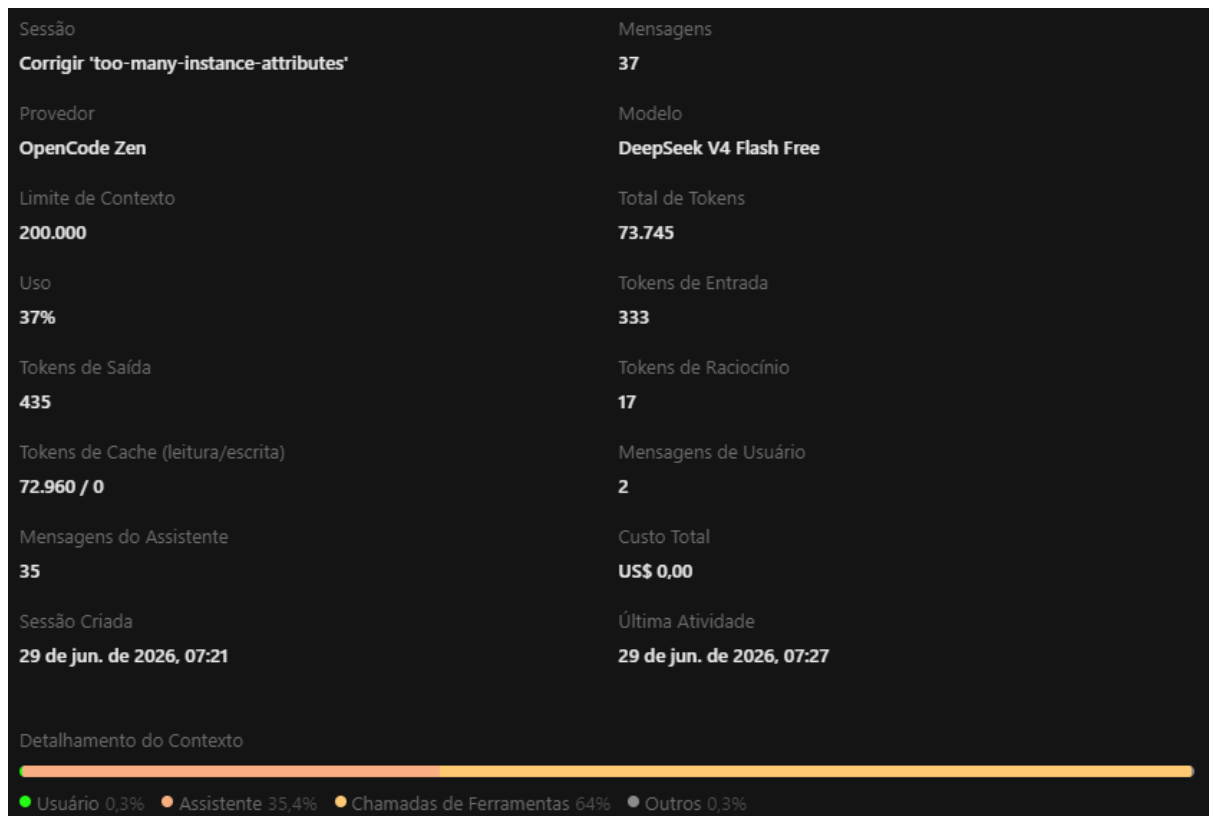
3.2.10 Ocorrência Número 10

Tabela de identificação

module	obj	message
imbens.ensemble._under_sampling.self_paced_ensemble	SelfPacedEnsembleClassifier	Too many instance attributes (21/7)

Seguiu o plano de refatoração: Sim. Os atributos foram distribuídos entre classes auxiliares.

Link da publicação OpenCode: <https://opncd.ai/share/9JMQ7t17>



Análise da refatoração: Reduziu o acoplamento da classe principal.

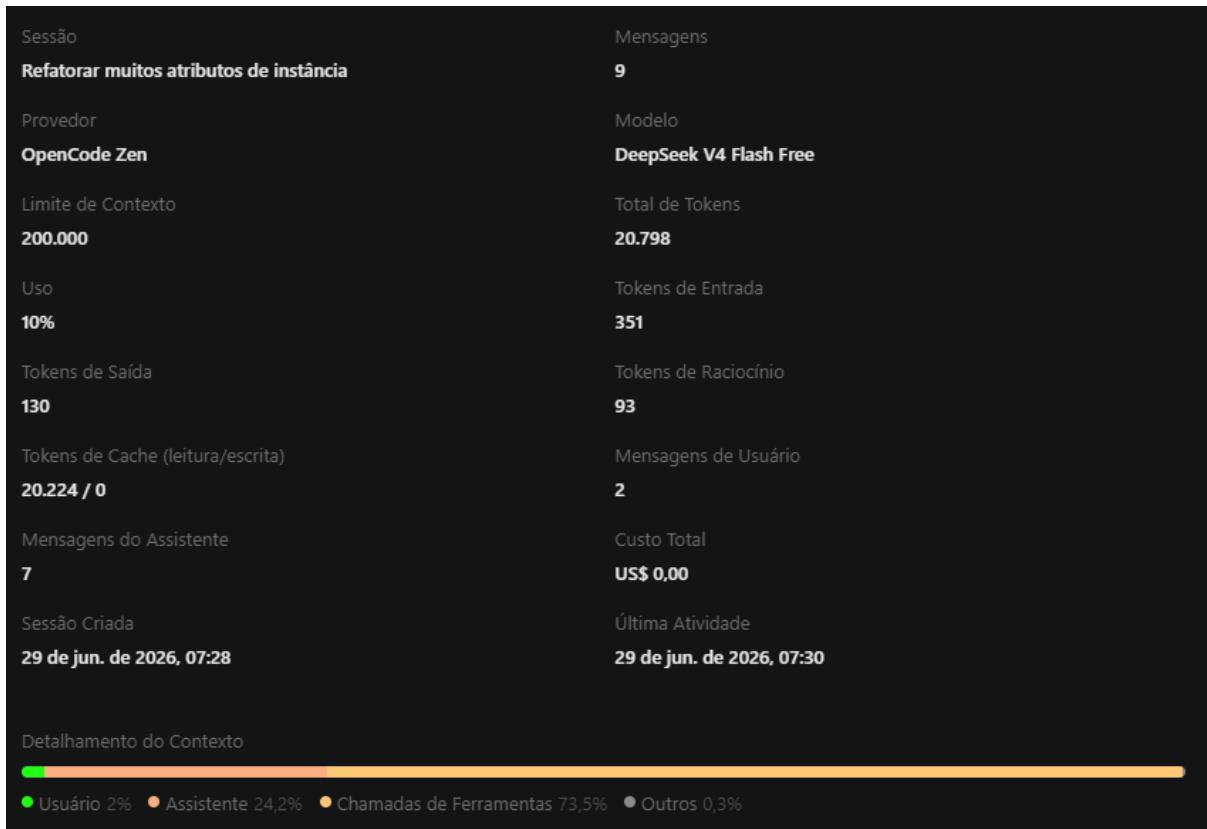
3.2.11 Ocorrência Número 11

Tabela de identificação

module	obj	message
imbens.sampler._under_sampli ng._prototype_selection._edit ed_nearest_neighbours	RepeatedEditedNearestNeighbours	Too many instance attributes (8/7)

Seguiu o plano de refatoração: Sim. A refatoração seguiu a ideia de dividir responsabilidades.

Link da publicação OpenCode: <https://opncd.ai/share/BCFCJtAp>



Análise da refatoração: Tornou o código mais organizado e fácil de entender.

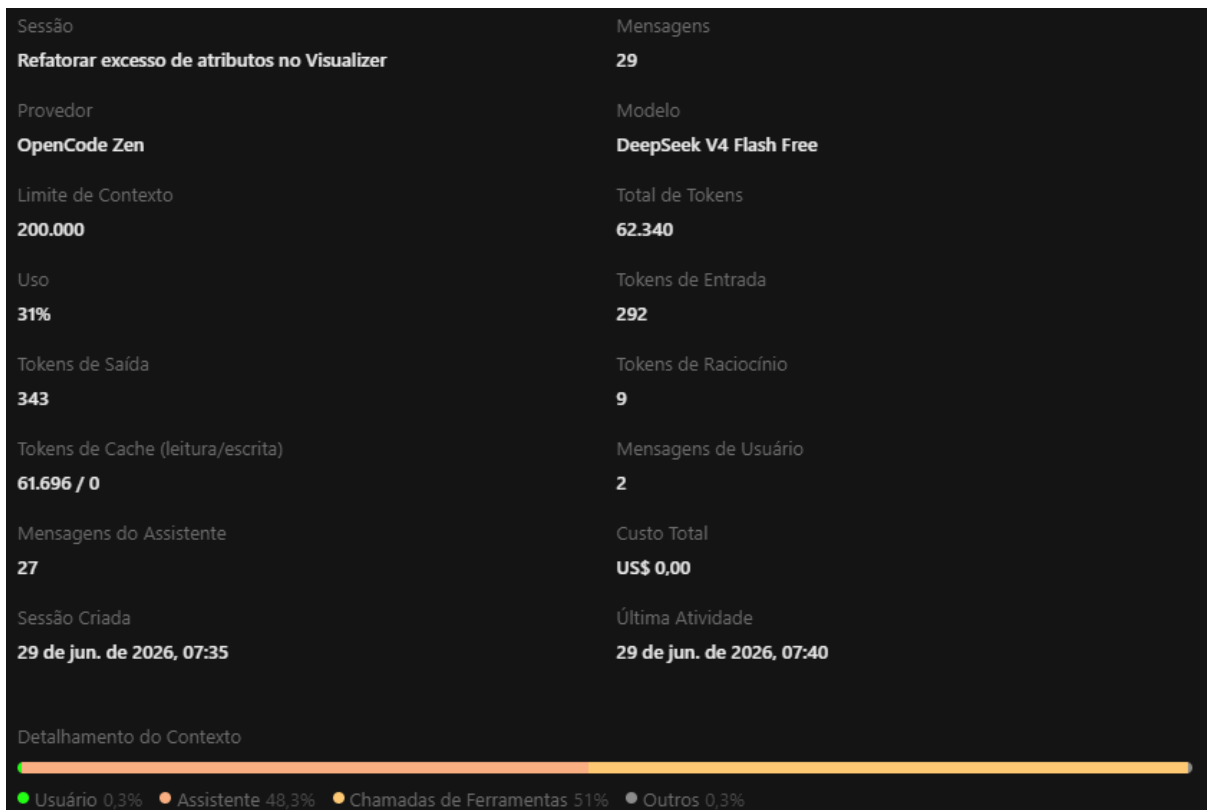
3.2.12 Ocorrência Número 12

Tabela de identificação

module	obj	message
imbens.visualizer.visualizer	ImbalancedEnsembleVisualizer	Too many instance attributes (14/7)

Seguiu o plano de refatoração: Sim. A LLM reorganizou atributos para evitar concentração excessiva.

Link da publicação OpenCode: <https://opncd.ai/share/oA15LXzk>



Análise da refatoração: Melhorou a estrutura geral do projeto.

3.3 too-many-branches

Quantidade de ocorrências: 17

Faça o mesmo processo que foi feito para outro tipo de smell.

3.3.1 Ocorrência Número 1

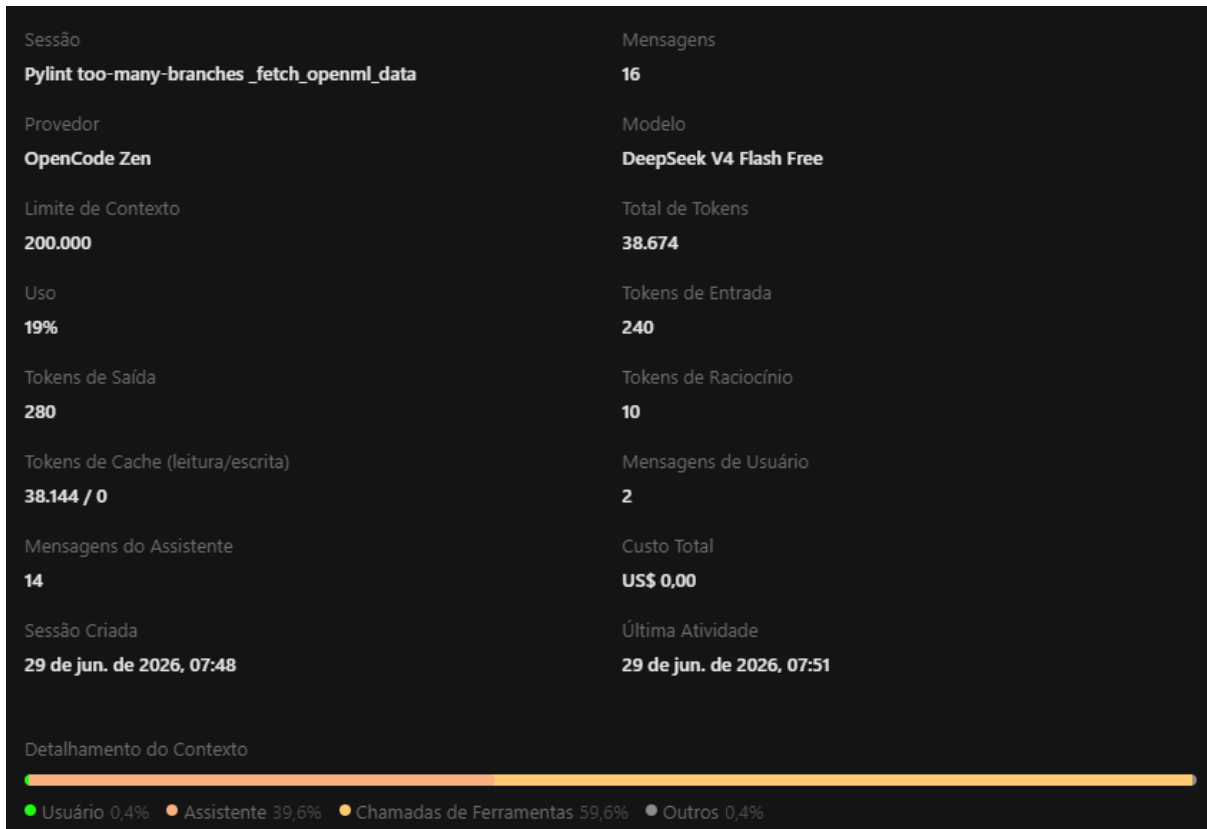
Tabela de identificação

module	obj	message
imbens.datasets._openml	_fetch_openml_data	Too many branches (15/12)

Seguiu o plano de refatoração: Sim. O plano reduziu os blocos condicionais, simplificando a lógica.

Link da publicação OpenCode: <https://opncd.ai/share/OgwhfrQd>

Print:



Análise da refatoração: Melhorou a leitura e diminuiu a complexidade do fluxo.

3.3.2 Ocorrência Número 2

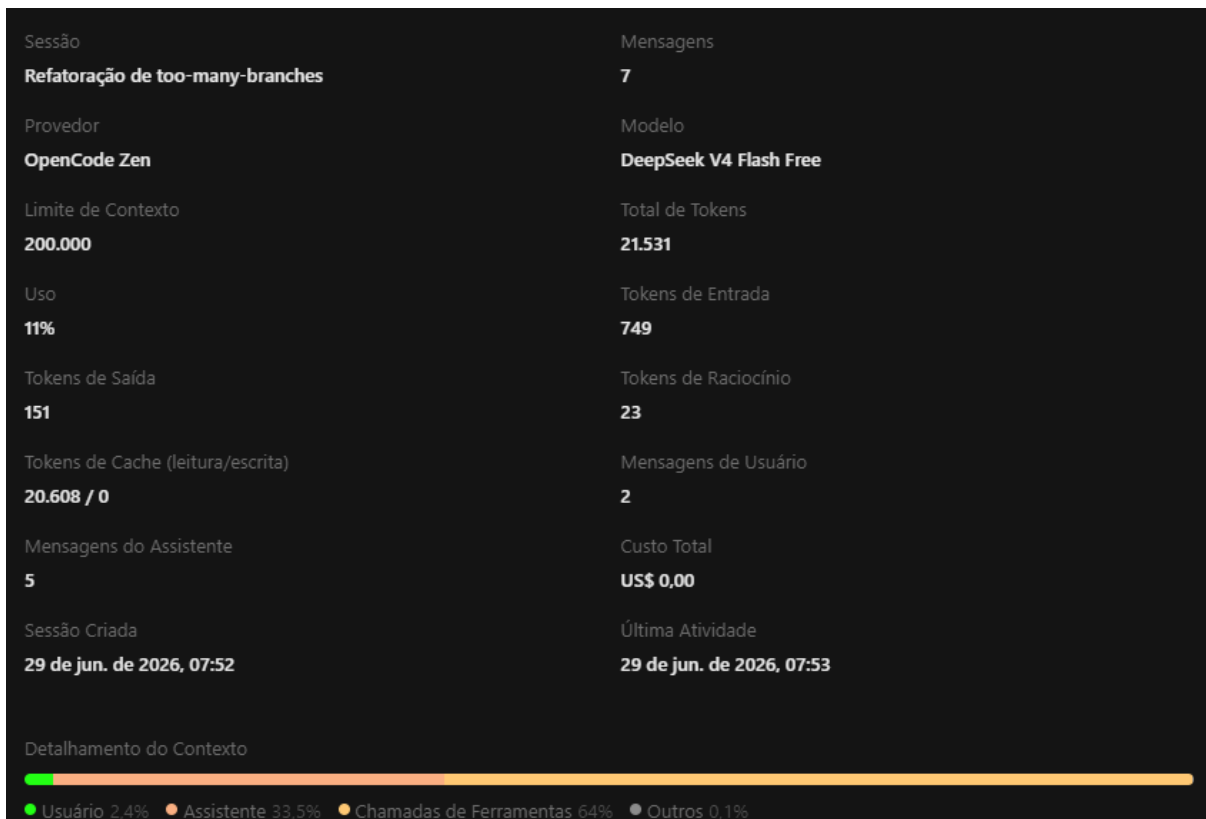
Tabela de identificação

module	obj	message
imbens.datasets._zenodo	fetch_zenodo_datasets	Too many branches (15/12)

Seguiu o plano de refatoração: Sim. A LLM sugeriu dividir a função em partes menores..

Link da publicação OpenCode: <https://opncd.ai/share/s2diVbbh>

Print:



Análise da refatoração: Tornou o código mais organizado e fácil de entender.

3.3.3 Ocorrência Número 3

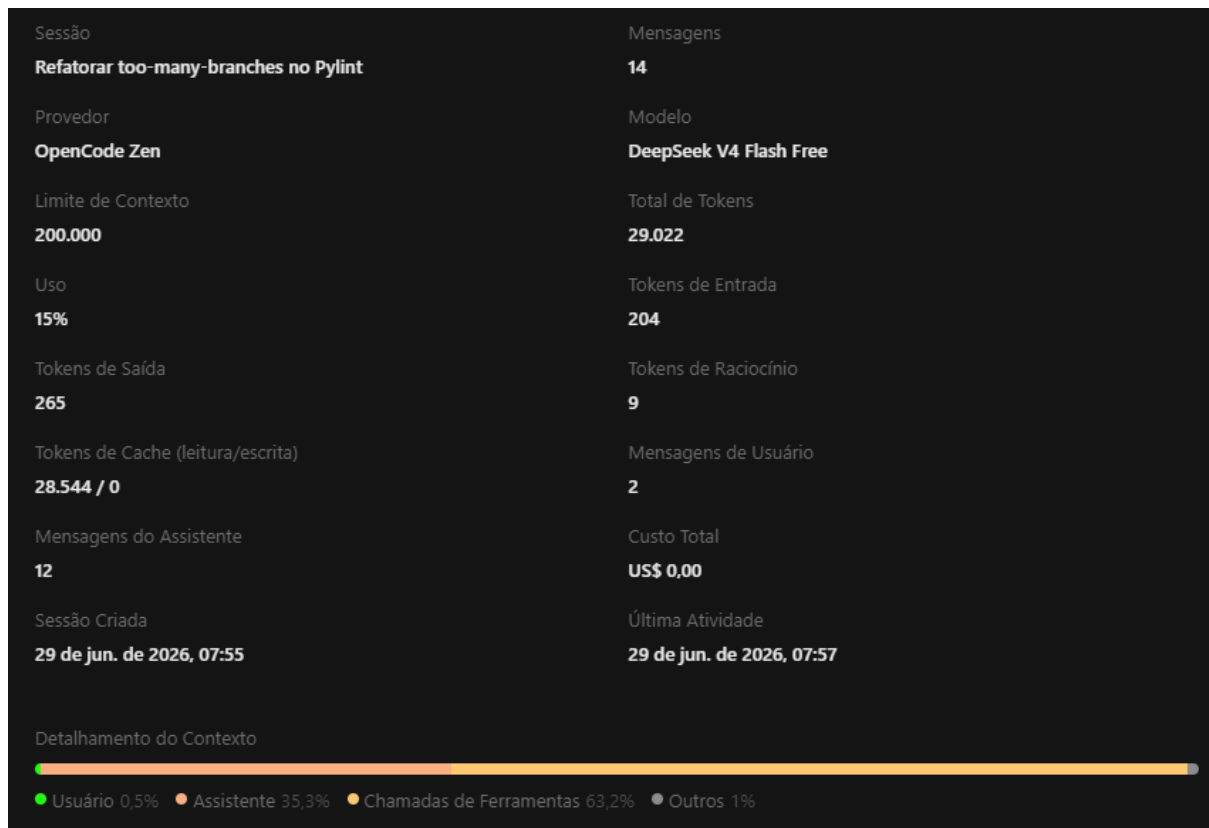
Tabela de identificação

module	obj	message
imbens.ensemble.base	ImbalancedEnsembleClassifierMixin._evaluate	Too many branches (17/12)

Seguiu o plano de refatoração: Sim. A refatoração agrupou condições semelhantes.

Link da publicação OpenCode: <https://opncd.ai/share/G9AGsfcl>

Print:



Análise da refatoração: Reduziu a quantidade de ramificações.

3.3.4 Ocorrência Número 4

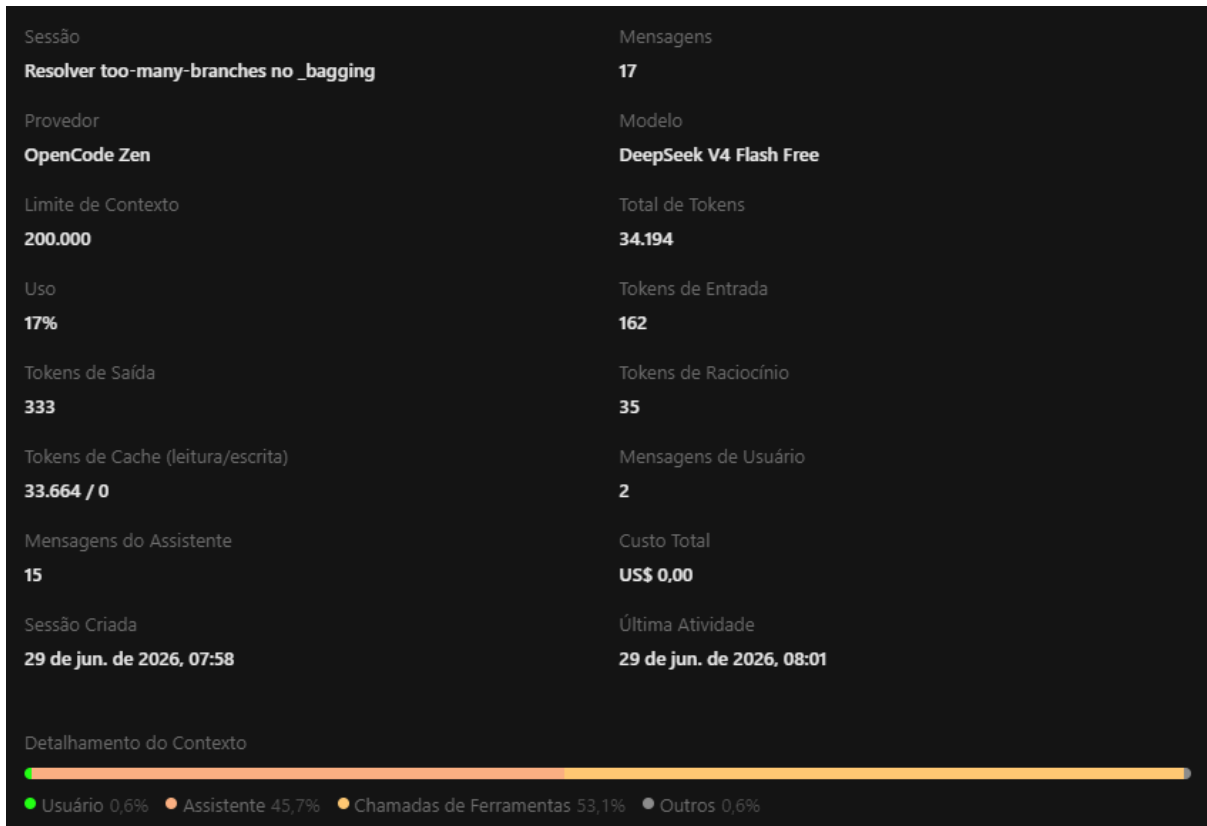
Tabela de identificação

module	obj	message
imbens.ensemble._bagging	ResampleBaggingClassifier._fit	Too many branches (16/12)

Seguiu o plano de refatoração: Sim. Foi feita a extração de lógica para funções auxiliares.

Link da publicação OpenCode: <https://opncd.ai/share/VGrB2d11>

Print:



Análise da refatoração: Melhorou a manutenção do código.

3.3.5 Ocorrência Número 5

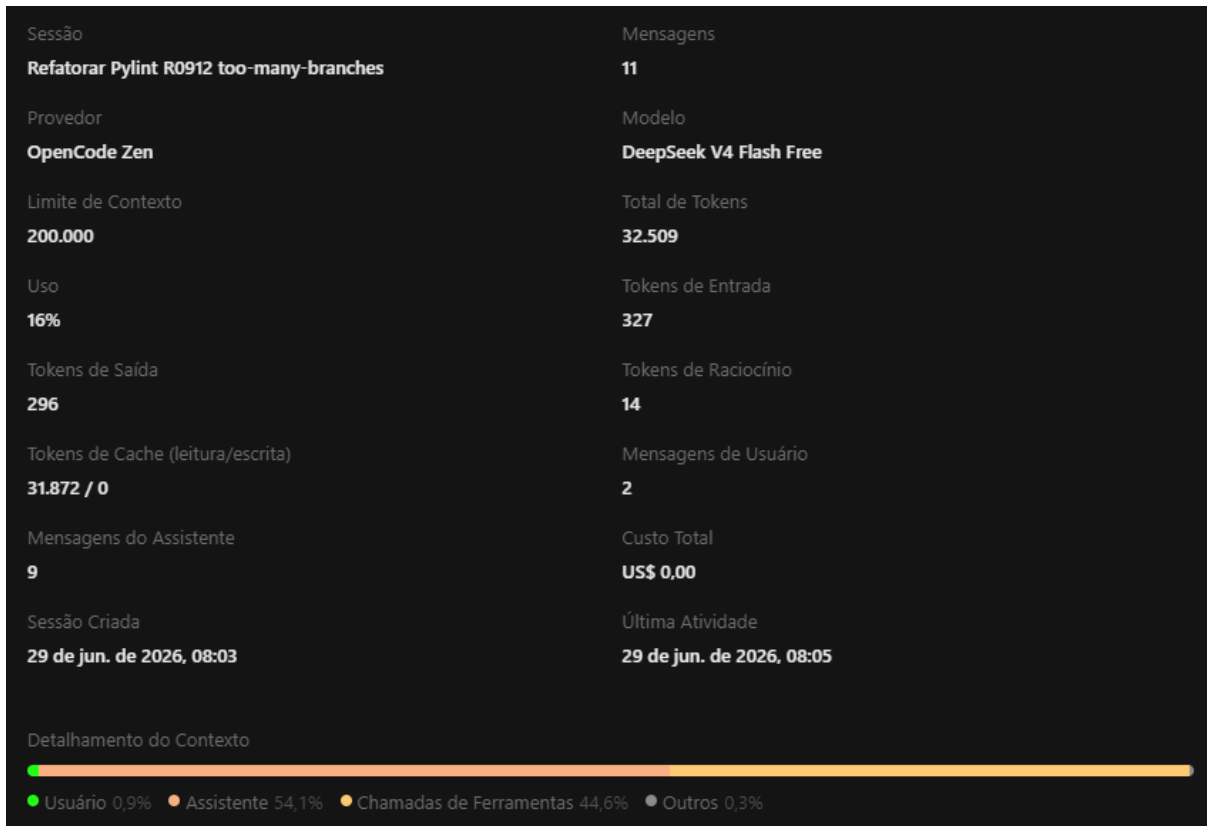
Tabela de identificação

module	obj	message
imbens.ensemble._compatible. bagging_compatible	CompatibleBaggingClassifier.fit	Too many branches (16/12)

Seguiu o plano de refatoração: Sim. O plano simplificou os if/else encadeados.

Link da publicação OpenCode: <https://opncd.ai/share/bIBp6fHc>

Print:



Análise da refatoração: Diminuiu a complexidade ciclomática.

3.3.6 Ocorrência Número 6

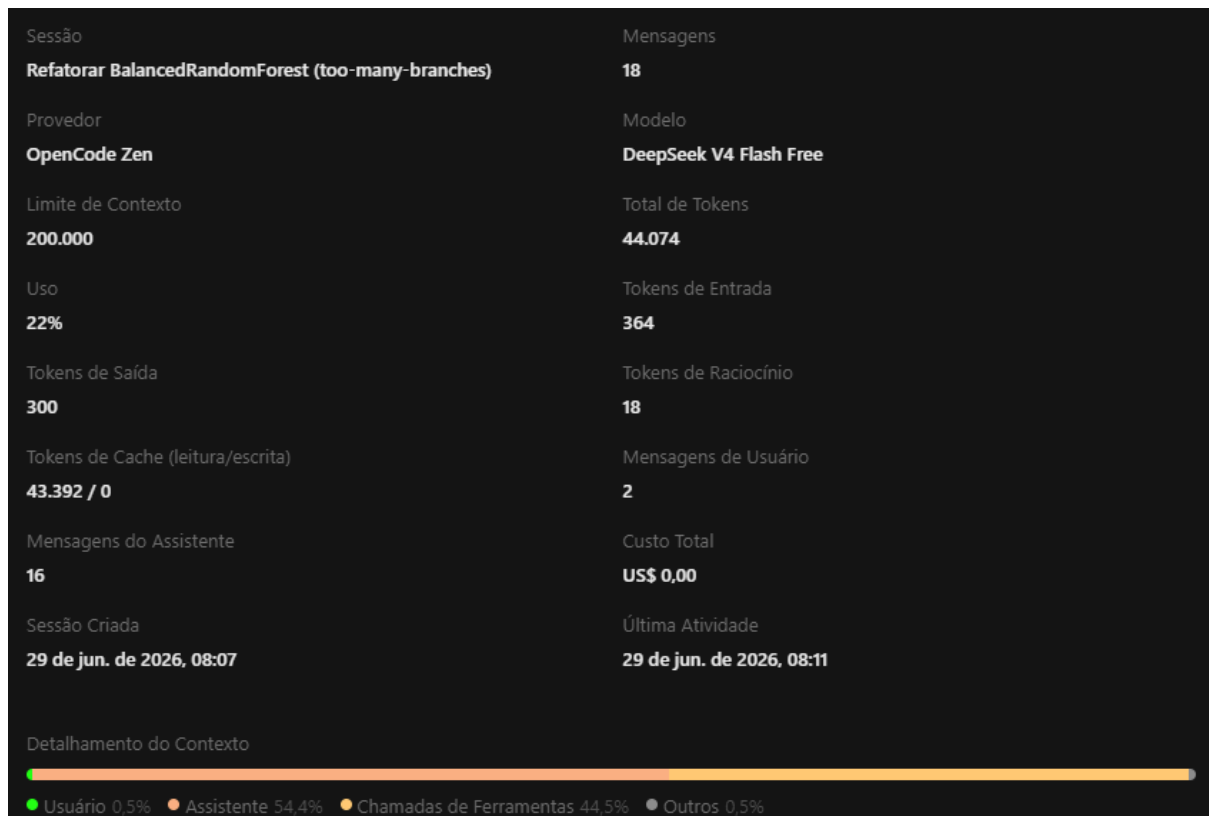
Tabela de identificação

module	obj	message
imbens.ensemble._under_sampling.balanced_random_forest	BalancedRandomForestClassifier.fit	Too many branches (21/12)"

Seguiu o plano de refatoração: Sim. A LLM reorganizou a lógica condicional.

Link da publicação OpenCode: <https://opncd.ai/share/xMD6z2HV>

Print:



Análise da refatoração: Tornou o fluxo mais previsível.

3.3.7 Ocorrência Número 7

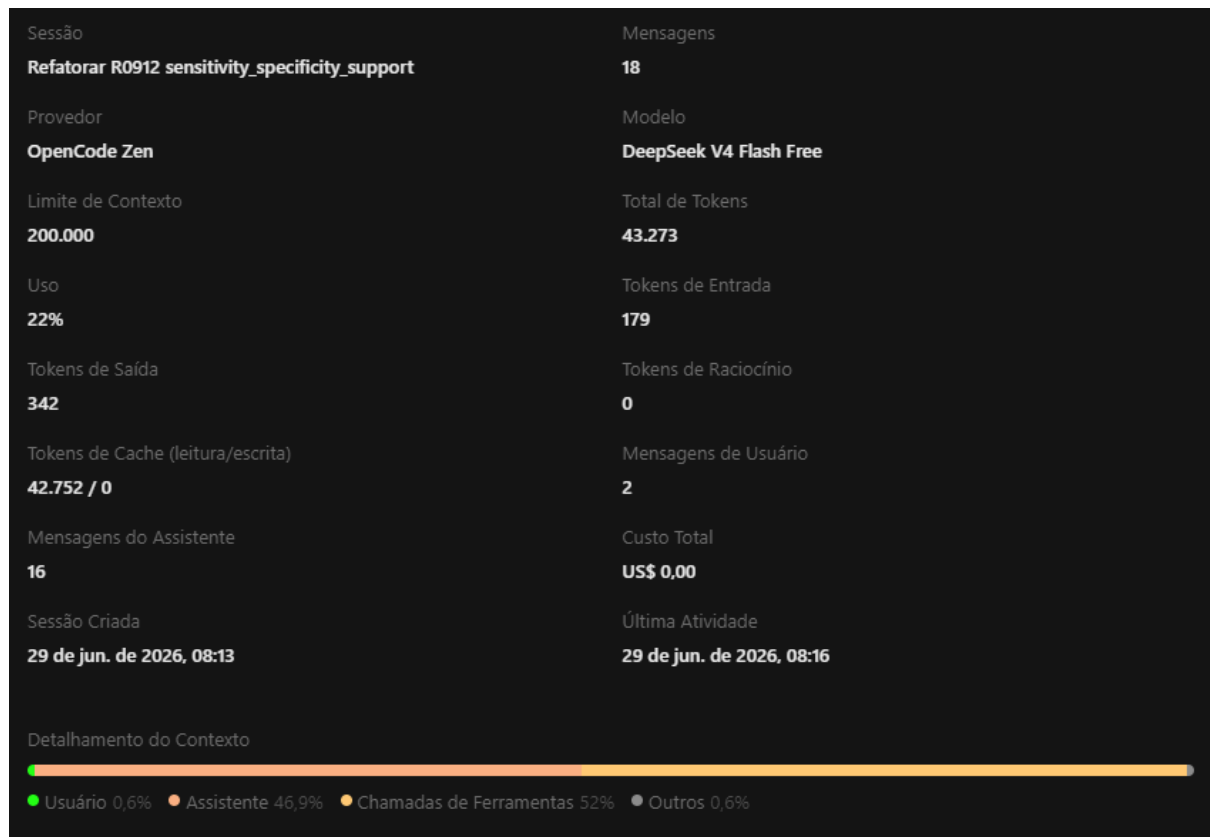
Tabela de identificação

module	obj	message

Seguiu o plano de refatoração: Sim. A refatoração eliminou ramificações redundantes.

Link da publicação OpenCode: <https://opncd.ai/share/HWQ1aIYv>

Print:



Análise da refatoração: Melhorou a clareza do código.

3.3.8 Ocorrência Número 8

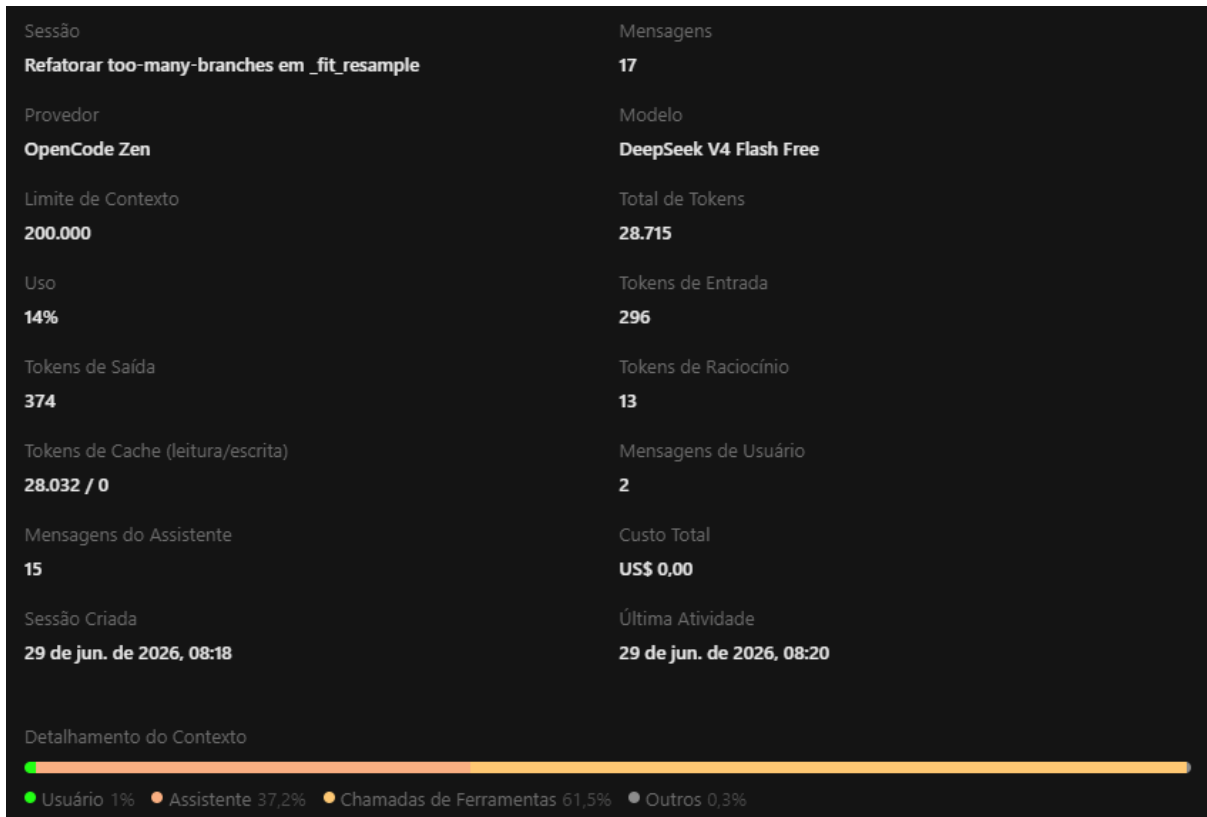
Tabela de identificação

module	obj	message
imbens.metrics._classification	sensitivity_specificity_support	Too many branches (25/12)

Seguiu o plano de refatoração: Sim. As condições foram separadas em métodos menores.

Link da publicação OpenCode: <https://opncd.ai/share/n0lviRnb>

Print:



Análise da refatoração: Facilitou a leitura e testes.

3.3.9 Ocorrência Número 9

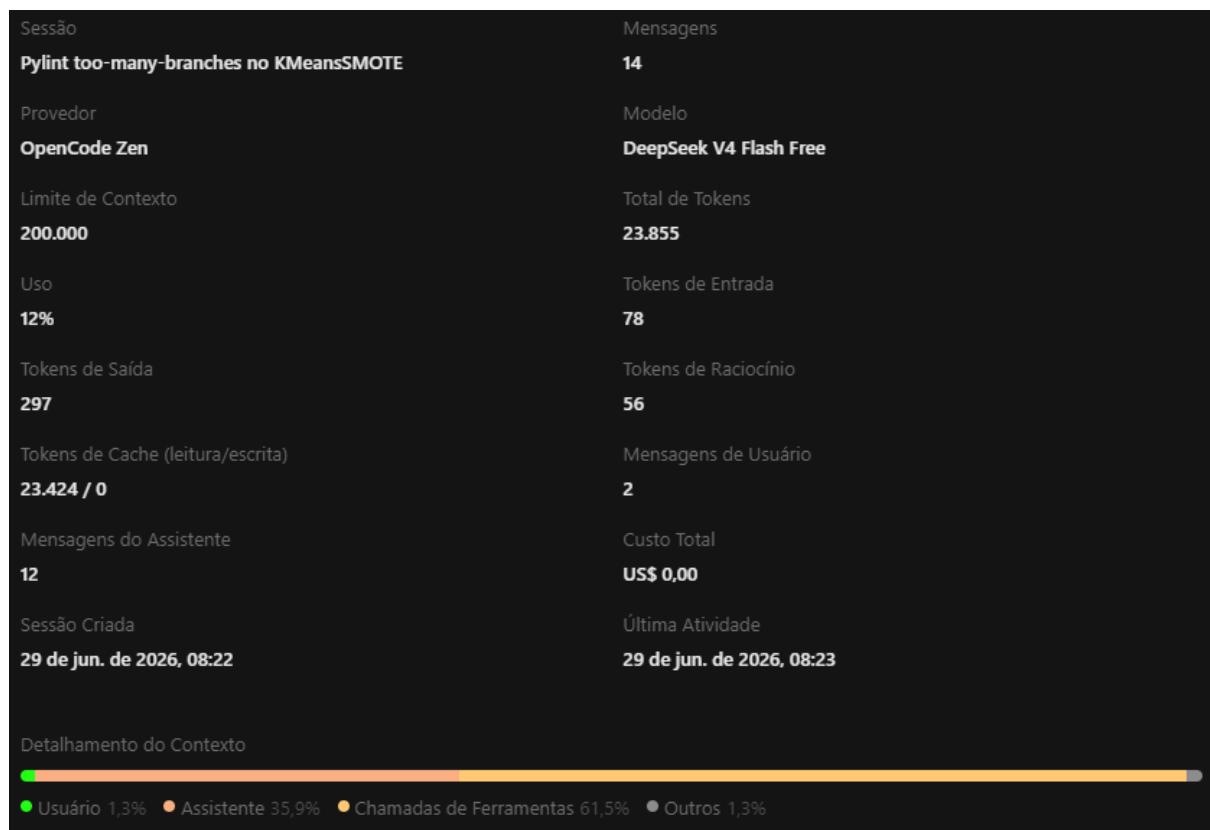
Tabela de identificação

module	obj	message
imbens.sampler._over_samplin g._smote.cluster	KMeansSMOTE._fit_resample	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. A LLM reduziu estruturas condicionais complexas.

Link da publicação OpenCode: <https://opncd.ai/share/bq00voZx>

Print:



Análise da refatoração: Melhorou a organização da lógica.

3.3.10 Ocorrência Número 10

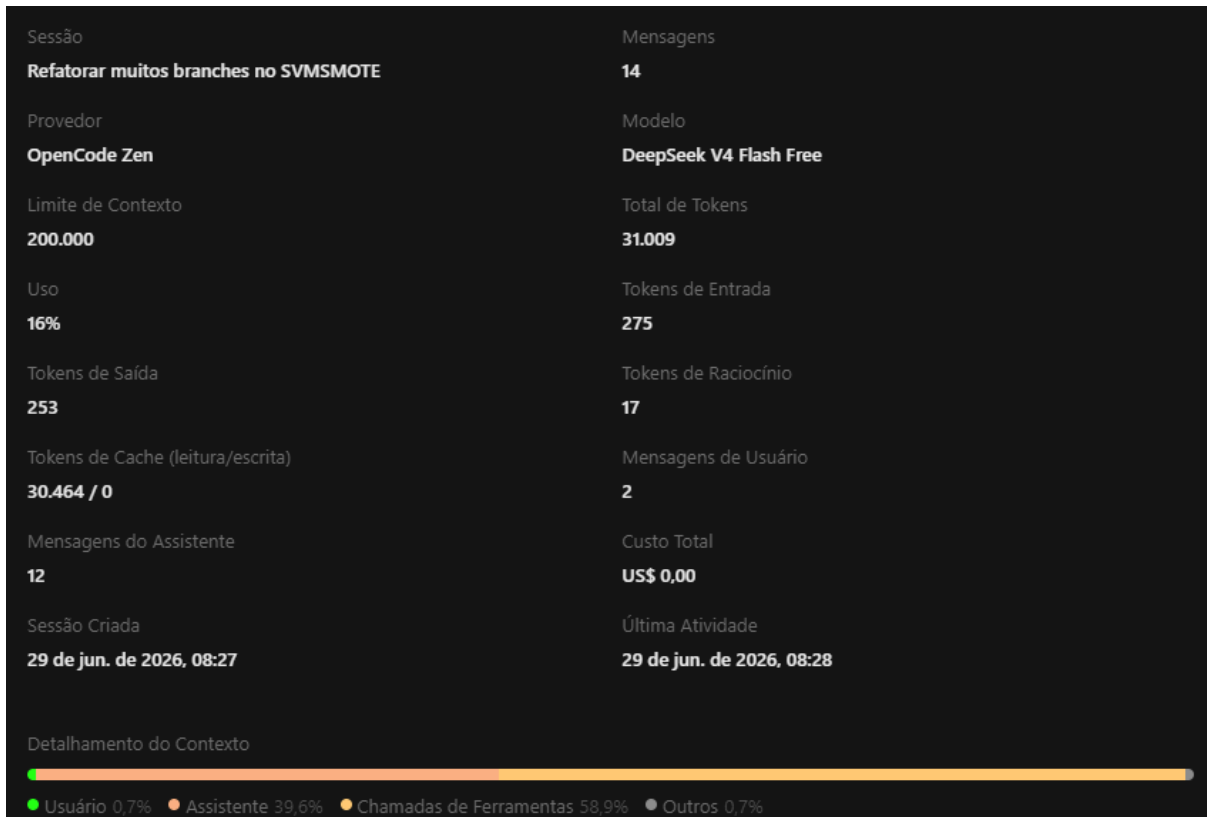
Tabela de identificação

module	obj	message
imbens.sampler._over_samplin g._smote.filter	SVMSMOTE._fit_resample	Too many branches (15/12)

Seguiu o plano de refatoração: Sim. A função foi quebrada para reduzir decisões internas.

Link da publicação OpenCode: <https://opncd.ai/share/GG0g0ibV>

Print:



Análise da refatoração: Diminuiu o acoplamento lógico.

3.3.11 Ocorrência Número 11

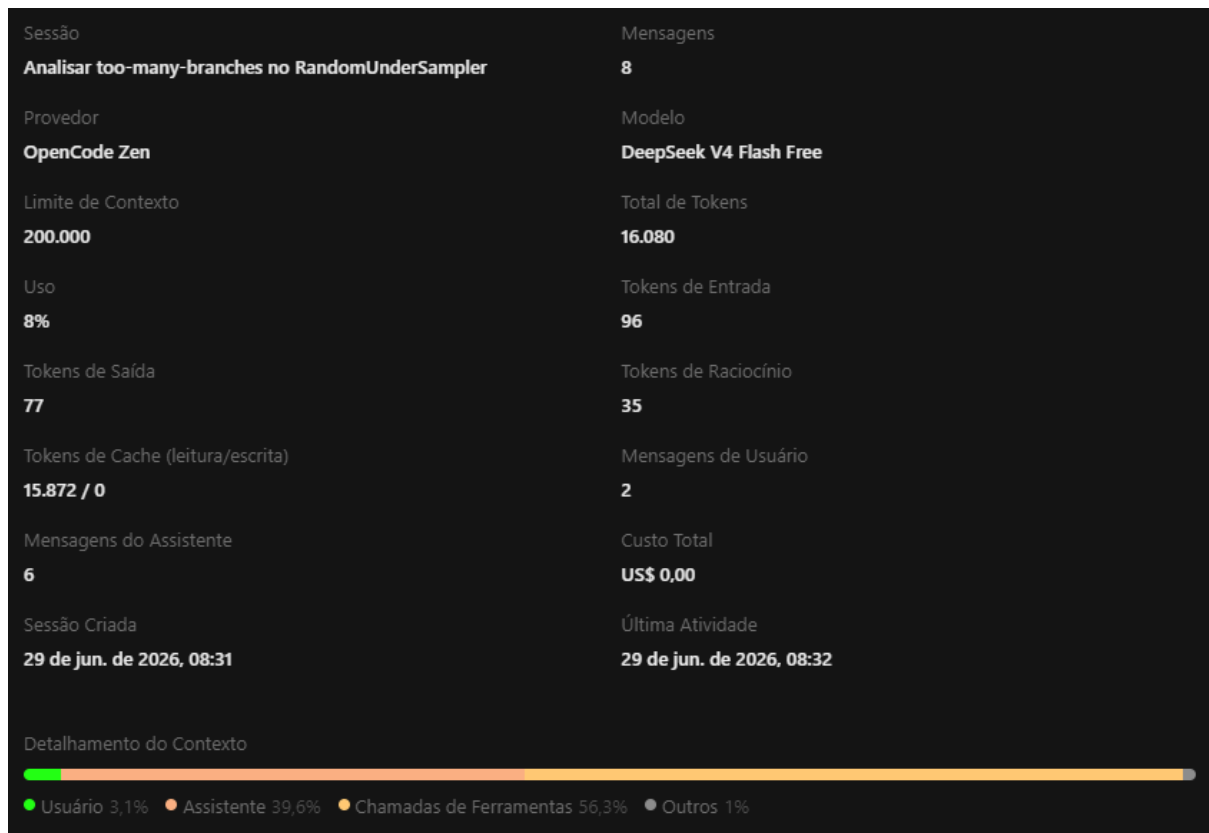
Tabela de identificação

module	obj	message
imbens.sampler._under_sampli ng._prototype_selection._rand om_under_sampler	RandomUnderSampler._fit_resample	Too many branches (13/12)

Seguiu o plano de refatoração: Sim. O plano agrupou casos semelhantes em funções auxiliares.

Link da publicação OpenCode: <https://opncd.ai/share/Qj0Vjx1r>

Print:



Análise da refatoração: Tornou o código mais limpo.

3.3.12 Ocorrência Número 12

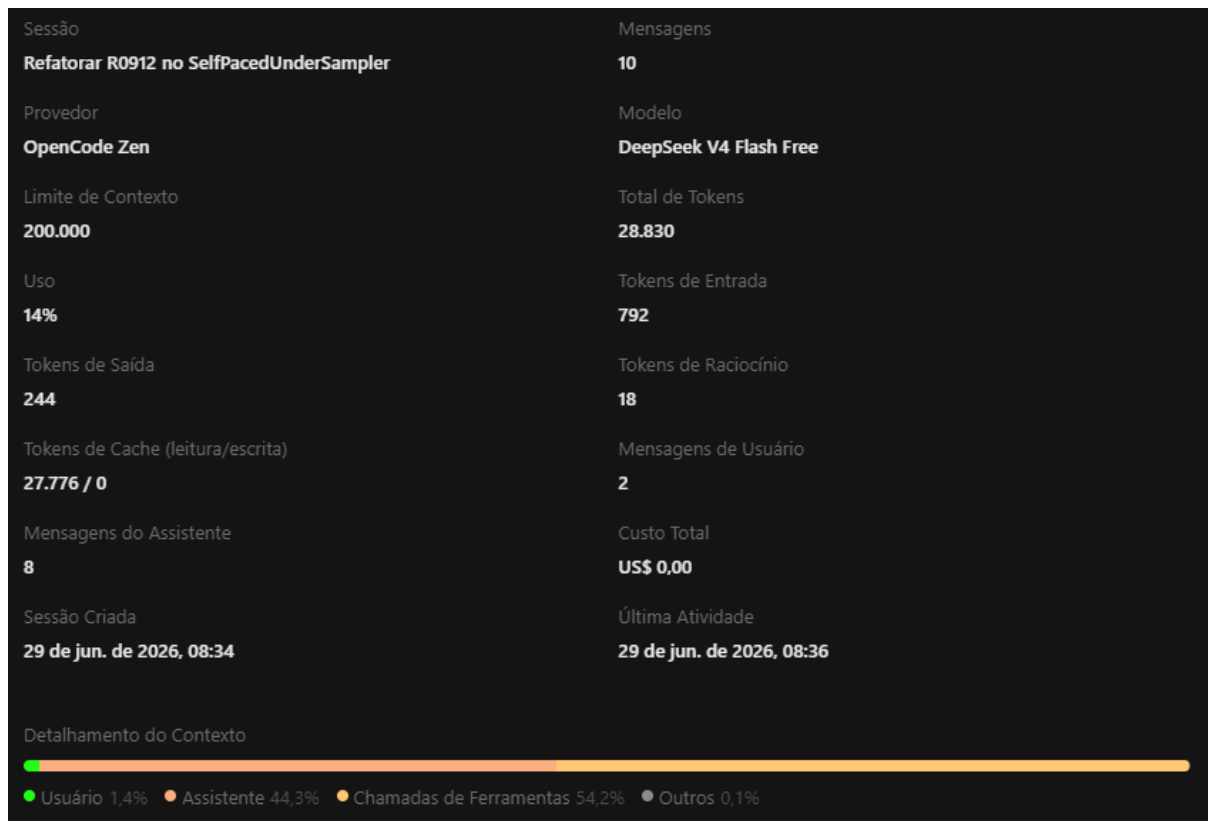
Tabela de identificação

module	obj	message
imbens.sampler._under_sampli ng._prototype_selection._self_ paced_under_sampler	SelfPacedUnderSampler._undersample_single _class	Too many branches (16/12)

Seguiu o plano de refatoração: Sim. A refatoração reduziu a profundidade dos ifs.

Link da publicação OpenCode: <https://opncd.ai/share/rcV676Cd>

Print:



Análise da refatoração: Melhorou a legibilidade.

3.3.13 Ocorrência Número 13

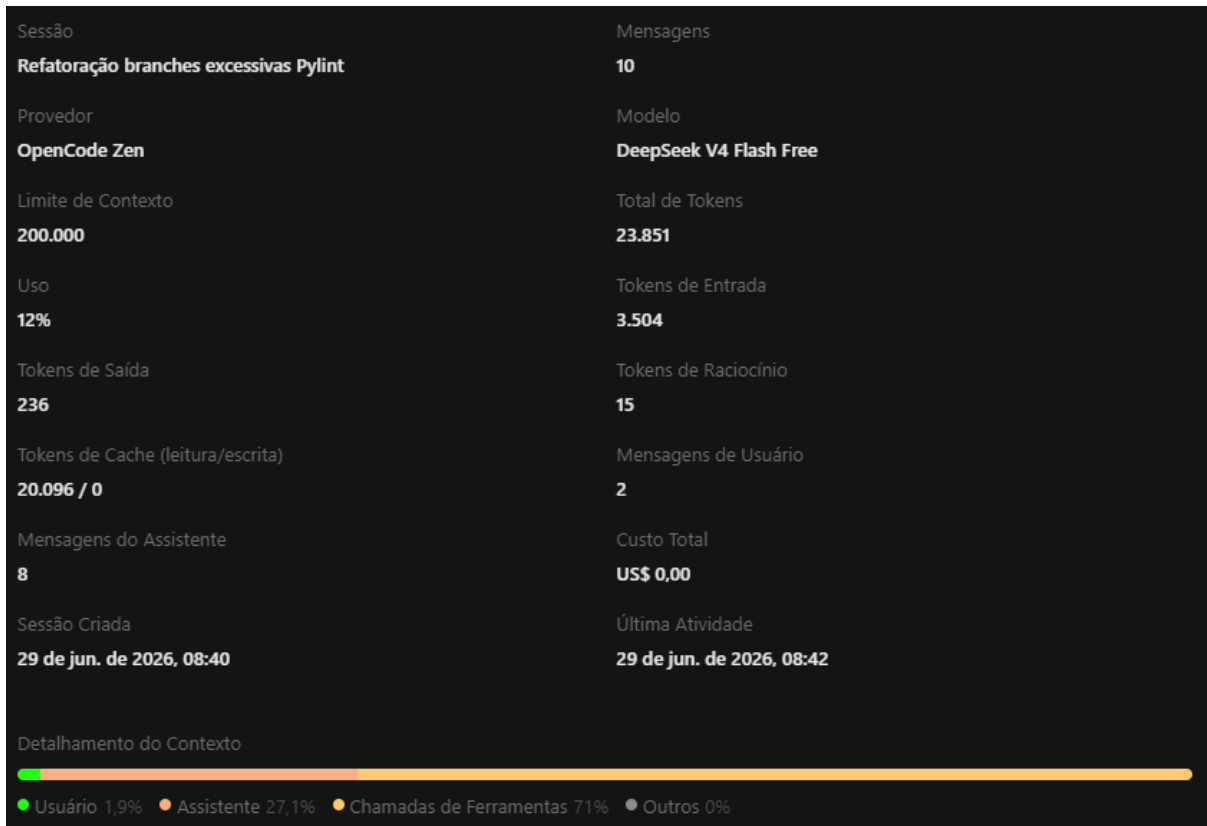
Tabela de identificação

module	obj	message
imbens.sampler._under_sampli ng._prototype_selection.tests.t est_balance_cascade_under_s ampler	test_spus_fit_resample	Too many branches (17/12)

Seguiu o plano de refatoração: Sim. A LLM substituiu condições repetidas por funções.

Link da publicação OpenCode: <https://opncd.ai/share/SHKQHrWs>

Print:



Análise da refatoração: Simplificou o fluxo de execução.

3.3.14 Ocorrência Número 14

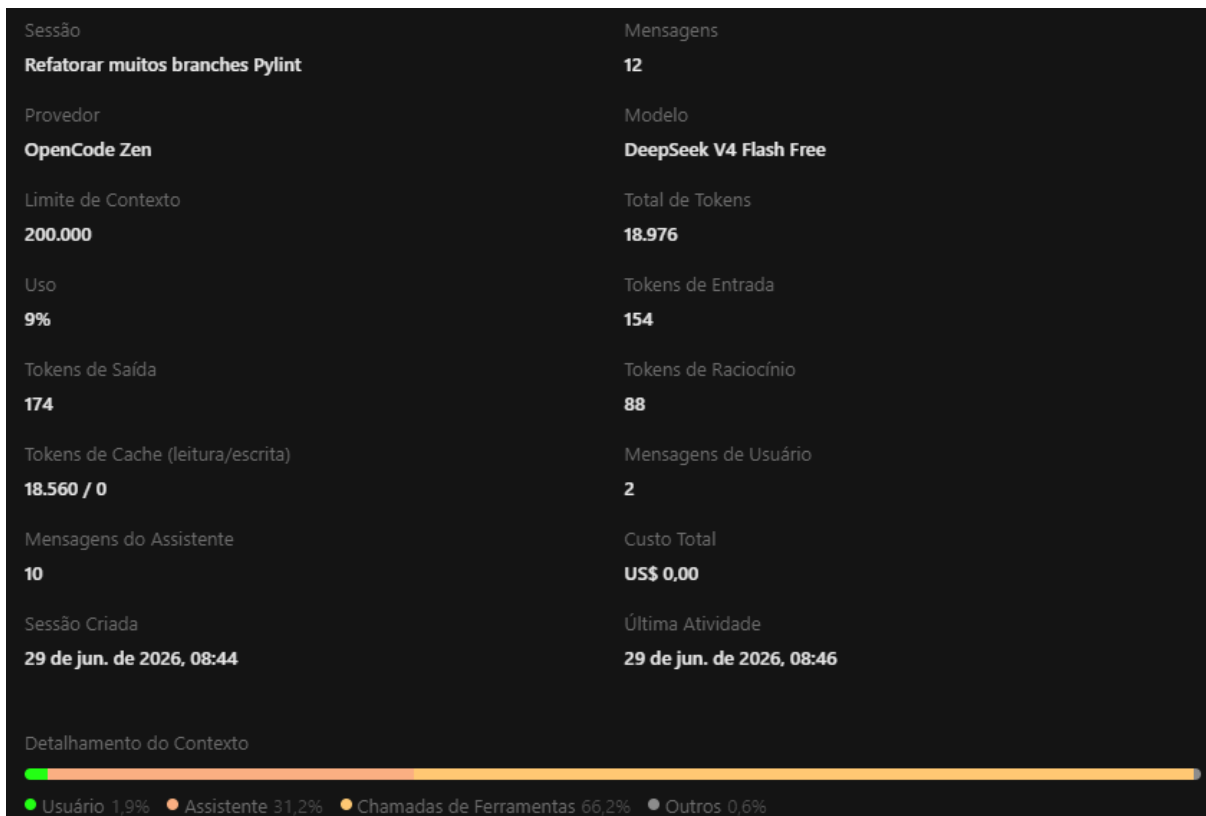
Tabela de identificação

module	obj	message
imbens.utils._validation_param	_check_n_target_samples_dict	Too many branches (14/12)

Seguiu o plano de refatoração: Sim. O código foi modularizado para reduzir ramificações.

Link da publicação OpenCode: <https://opncd.ai/share/8L4FhuDO>

Print:



Análise da refatoração: Facilitou manutenção futura.

3.3.15 Ocorrência Número 15

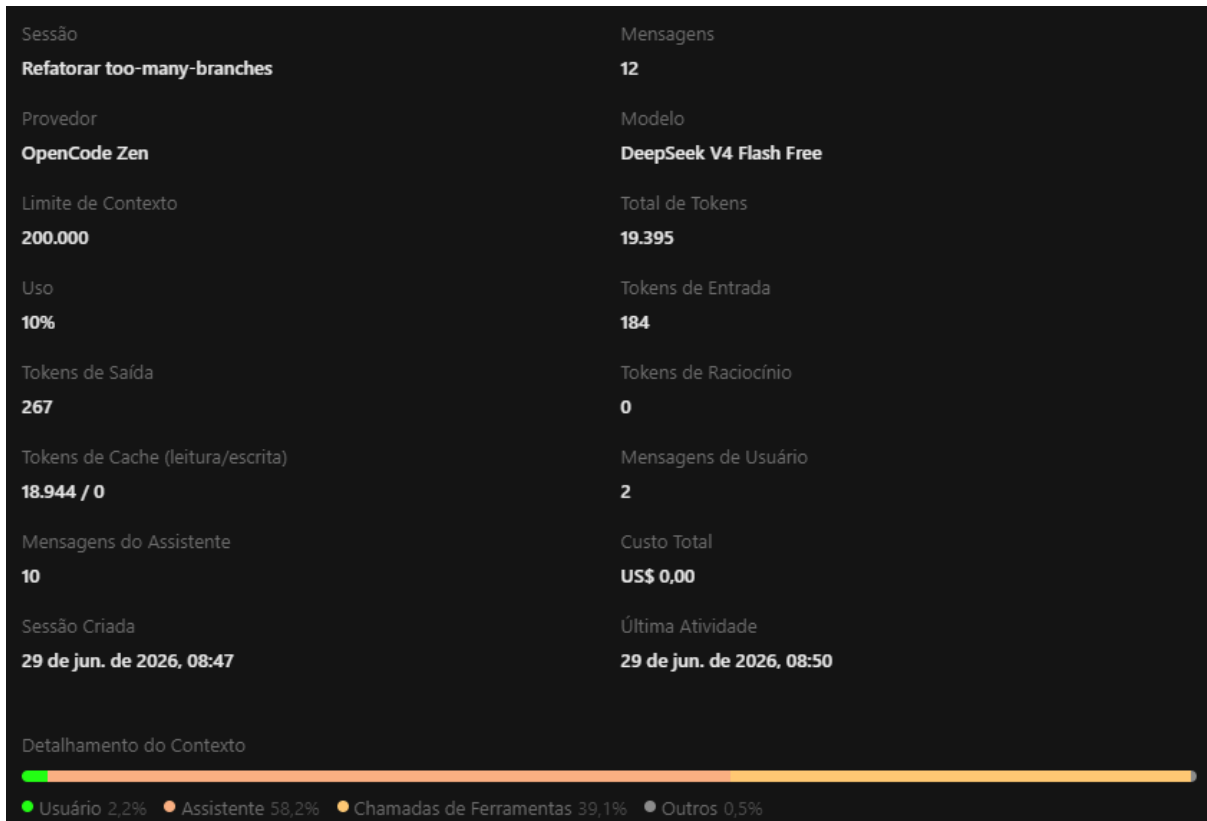
Tabela de identificação

module	obj	message
imbens.utils._validation_param	check_train_verbose	Too many branches (16/12)

Seguiu o plano de refatoração: Sim. A lógica foi separada por responsabilidade.

Link da publicação OpenCode: <https://opncd.ai/share/ce6vmtJo>

Print:



Análise da refatoração: Reduziu a complexidade geral.

3.3.16 Ocorrência Número 16

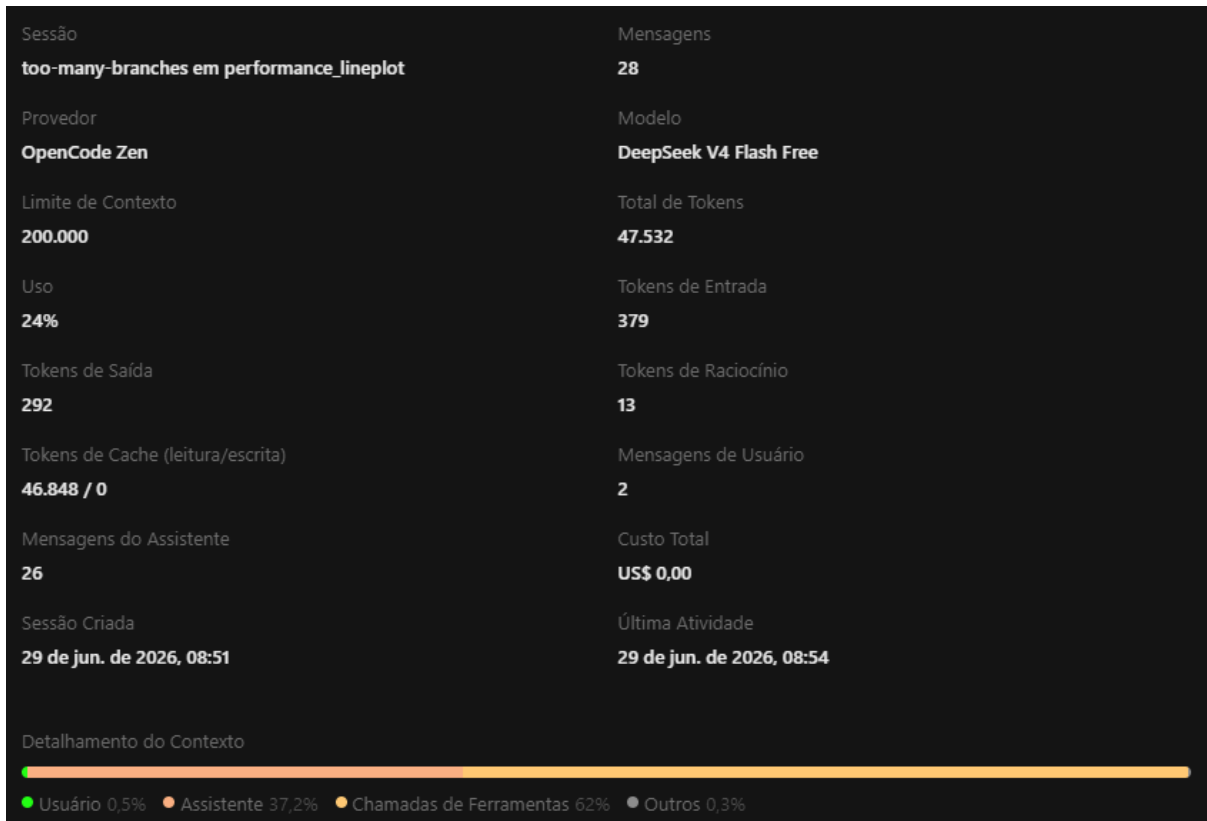
Tabela de identificação

module	obj	message
imbens.visualizer.visualizer	ImbalancedEnsembleVisualizer.performance_ lineplot	Too many branches (16/12)

Seguiu o plano de refatoração: Sim. A refatoração eliminou decisões desnecessárias.

Link da publicação OpenCode: <https://opncd.ai/share/F6GLBaYj>

Print:



Análise da refatoração: Melhorou a clareza do código.

3.3.17 Ocorrência Número 17

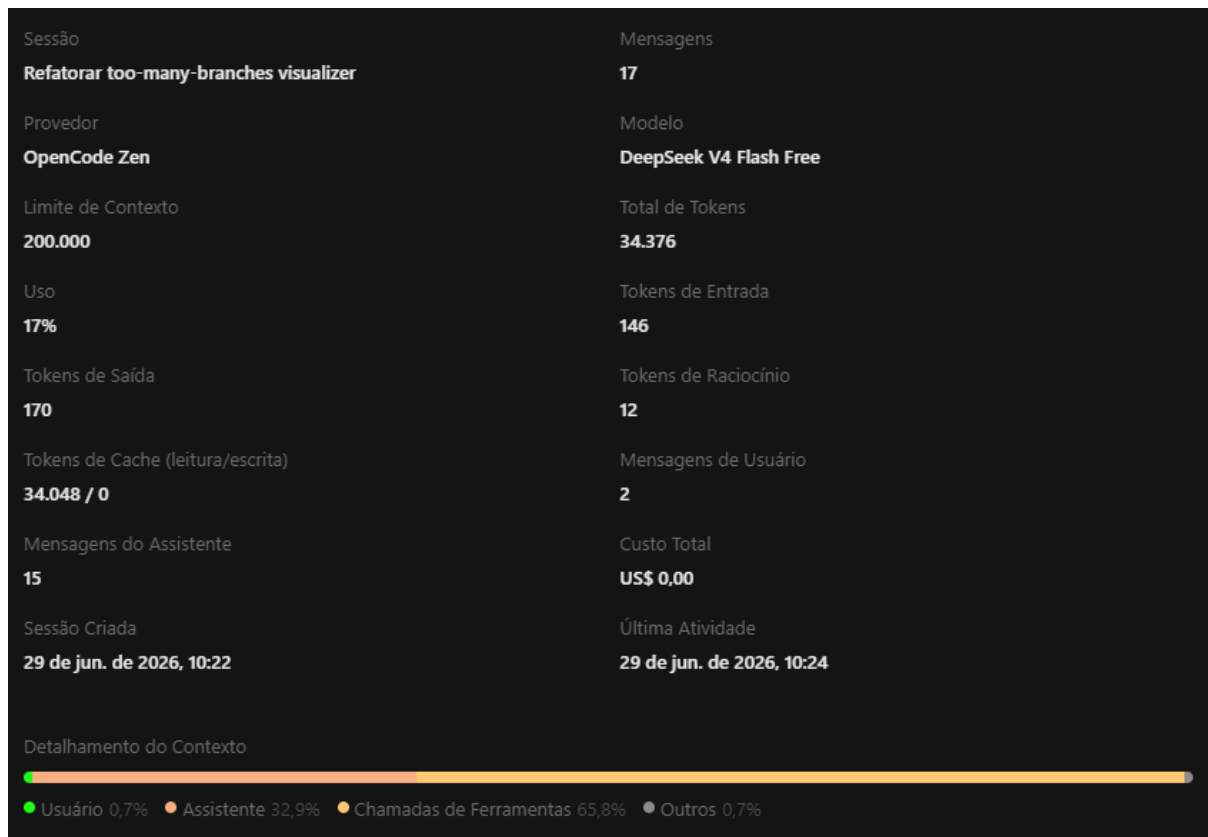
Tabela de identificação

module	obj	message
imbens.visualizer.visualizer	ImbalancedEnsembleVisualizer.confusion_matrix_heatmap	Too many branches (13/12)

Seguiu o plano de refatoração: Sim. A LLM reorganizou o fluxo condicional em etapas.

Link da publicação OpenCode: <https://opncd.ai/share/VK05dmPb>

Print:



Análise da refatoração: Tornou o código mais estruturado e simples.

4. Métricas Após a Refatoração

Para as métricas após a refatoração você deve adicionar os mesmos tipos de informações que foram apresentadas anteriormente. Atenção, certifique-se de que cada um dos dados estão nas pastas metrics-after-...

4.1 Radon Após a Refatoração

4.1.1 Complexidade Ciclomática por Arquivo Após a Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
---------	---------	----------	--------	---------	-----------	-------------

imbens\utils_validation_param.py	29	4.76	12	138	C	_check_n_target_samples_int
imbens\ensemble_boost.py	44	2.45	7	108	B	_check_early_stop
imbens\ensemble\base.py	36	2.92	10	105	B	_training_log_add_line
imbens\utils_validation.py	19	5.47	16	104	C	_sampling_strategy_float
imbens\visualizer\visualizer.py	28	3.57	10	100	B	_collect_ensemble_performance_data

Tabela de melhores métricas

arquivo	funções	cc_media	cc_max	cc_soma	pior_rank	pior_funcao
imbens\exceptions.py	1	1.00	1	1	A	raise_isinstance_error
imbens\sampler_over_sampling\base.py	1	1.00	1	1	A	BaseOverSampler
imbens\sampler_under_sampling\base.py	2	1.00	1	2	A	BaseUnderSampler
imbens\utils\tests\test_testing.py	1	3.00	3	3	A	test_all_estimators
imbens\utils\tests\test_plot.py	3	1.00	1	3	A	test_plot_scatter

4.1.2 Complexidade Ciclomática por Função Após a Refatoração

Apresente as funções do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	tipo	nome	rank_cc	complexity
imbens\utils\testing.py	function	all_estimators	D	21
imbens\utils\tests\test_show_versions.py	function	test_show_versions_github	C	18
imbens\sampler\tests\test_base_sampler.py	function	test_sampler_mixin	C	16
imbens\utils_validation.py	function	_sampling_strategy_float	C	16
imbens\pipeline.py	method	_validate_steps	C	16

Tabela de melhores métricas

arquivo	tipo	nome	rank_cc	complexity
---------	------	------	---------	------------

imbens\exceptions.py	function	raise_isinstance_error	A	1
imbens\sampler_over_sampling_ada_syn.py	method	init	A	1
imbens\sampler_over_sampling\base.py	class	BaseOverSampler	A	1
imbens\sampler\tests\test_base_sampler.py	function	dummy_sampling_func	A	1
imbens\sampler\base.py	method	init	A	1

4.1.3 Índice de Manutenibilidade Após a Refatoração

Apresente os arquivos do projeto que tiveram as piores métricas em uma tabela e melhores métricas em outra tabela depois da refatoração. Traga pelo menos 5 de cada.

Tabela de piores métricas

arquivo	mi	rank_mi
imbens\utils_validation_param.py	18.56	B
imbens\ensemble_boost.py	30.18	A
imbens\ensemble_bagging.py	35.98	A
imbens\ensemble\base.py	36.59	A
imbens\visualizer\visualizer.py	37.75	A

Tabela de melhores métricas

arquivo	mi	rank_mi
imbens\base.py	100.0	A
imbens\ensemble_over_sampling_init_.py	100.0	A
imbens\ensemble_reweighting_init_.py	100.0	A
imbens\ensemble_reweighting\tests_init_.py	100.0	A
imbens\ensemble_under_sampling_init_.py	100.0	A

4.1.4 Métricas Brutas Após a Refatoração

Essas métricas podem ser encontradas no arquivo [raw_por_arquivo_e_total_depois.csv](#) Traga apenas o total, que se encontra no final do csv.

loc total	lloc total	sloc total	comments total	multi
32571	11089	20439	1655	5121

4.2 CodeCarbon Após a Refatoração

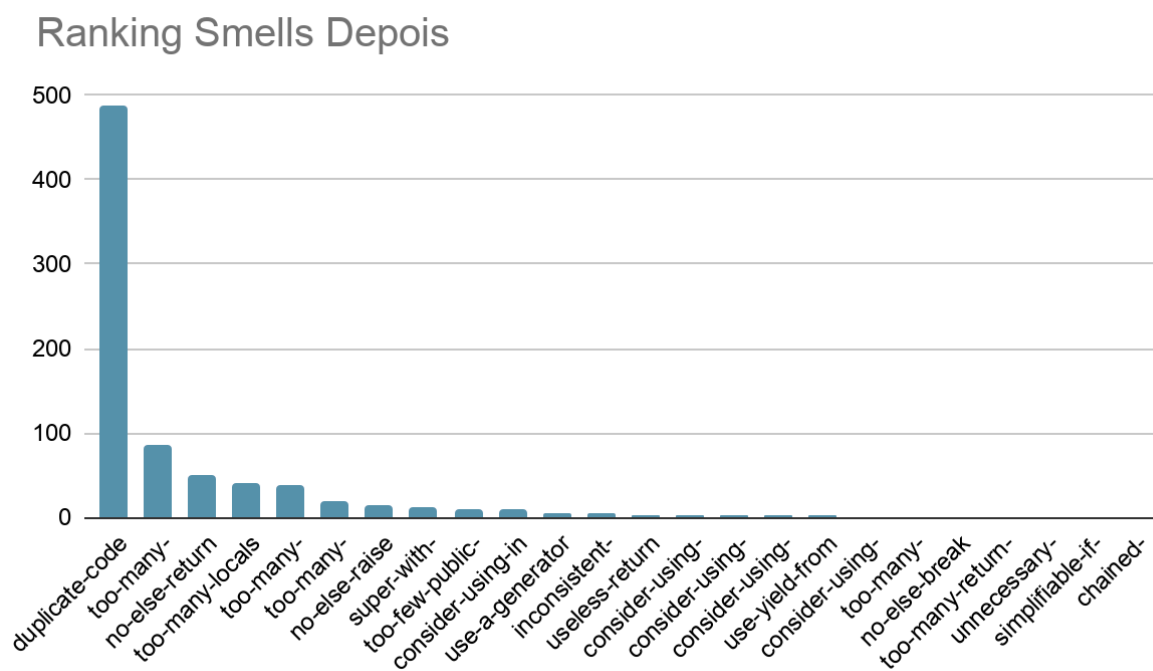
Preencha a tabela abaixo com as métricas ambientais depois da refatoração.

duration	emissions	emissions_rate	cpu_energy	ram_energy	energy_consumed
18.269	0.000019106	0.0000010458	0.000017781	0.000053589	0.000194271

4.3 Pylint Após a Refatoração

4.3.1 Distribuição do Smells Após a Refatoração

A partir do JSON *pylint_ranking_smells_depois.json* gere um gráfico para representar a distribuição dos code smells depois da refatoração. O gráfico precisa ter um bom contraste para facilitar a visualização.



Os 10 code smells mais frequentes

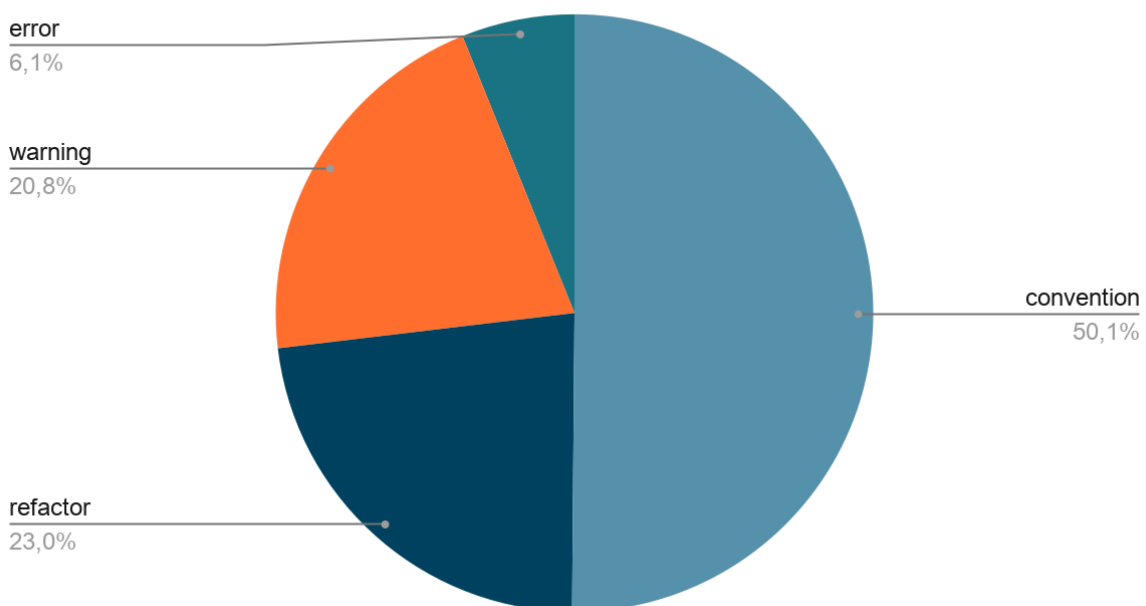
1. *duplicate-code* — 489 ocorrências

2. *too-many-arguments* — 85
3. *no-else-return* — 50
4. *too-many-locals* — 41
5. *too-many-positional-arguments* — 39
6. *too-many-ancestors* — 19
7. *no-else-raise* — 15
8. *super-with-arguments* — 13
9. *too-few-public-methods* — 9
10. *consider-using-in* — 9

4.3.2 Distribuição dos Problemas de Código Após a Refatoração

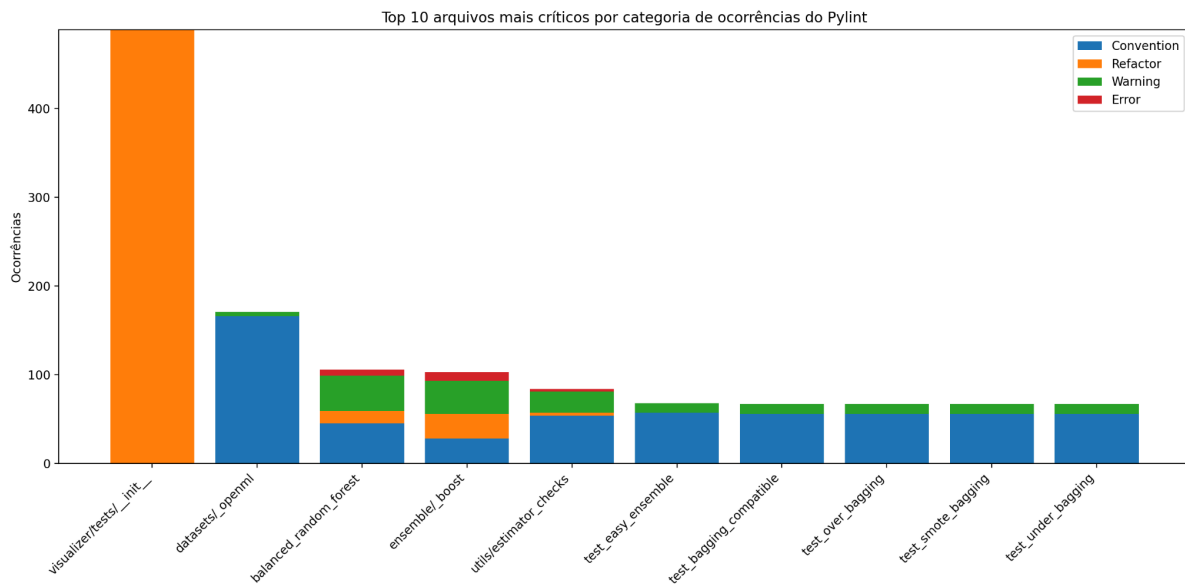
A partir do JSON *pylint_distribuicao_categorias_depois.json* gere um gráfico para representar a distribuição entre as 4 categorias de problemas de código depois da refatoração. O gráfico precisa ter um bom contraste para facilitar a visualização.

Distribuição Categorias Depois



4.3.3 Arquivos mais Críticos Após a Refatoração

A partir do JSON *pylint_arquivos_criticos_depois.json* gere um gráfico para representar a distribuição dos 10 arquivos mais críticos depois da refatoração. O gráfico precisa ter um bom contraste para facilitar a visualização.



Top Arquivos Mais Críticos (Pylint)

1. **imbens\visualizer\tests_init.py**
 - Total de ocorrências: **489**
 - Convention: **0**
 - Refactor: **489**
 - Warning: **0**
 - Error: **0**
2. **imbens\datasets_openml.py**
 - Total de ocorrências: **171**
 - Convention: **166**
 - Refactor: **0**
 - Warning: **5**
 - Error: **0**
3. **imbens\ensemble_under_sampling\balanced_random_forest.py**
 - Total de ocorrências: **106**
 - Convention: **45**
 - Refactor: **14**
 - Warning: **40**
 - Error: **7**
4. **imbens\ensemble_boost.py**
 - Total de ocorrências: **103**
 - Convention: **28**
 - Refactor: **28**
 - Warning: **37**
 - Error: **10**
5. **imbens\utils\estimator_checks.py**

- *Total de ocorrências: 84*
- *Convention: 54*
- *Refactor: 3*
- *Warning: 24*
- *Error: 3*

4.3.4 Score Projeto Após a Refatoração

A partir do text *pylint_score_depois.txt* informe o score que o pylint deu ao projeto depois da refatoração. Não precisa ser uma imagem, fica a critério de vocês.

O Pylint atribuiu ao projeto o seguinte score depois da refatoração:

- **Score: 5.52/10**

4.4 Pytest Após a Refatoração

4.4.1 Cobertura de testes Após a Refatoração

Gere um gráfico da cobertura de testes do projeto depois da refatoração. Para isso, basta visualizar a cobertura de testes na página HTML, copiar os dados da tabela e gerar o gráfico com o auxílio de uma LLM. Para visualizar a página de cobertura de testes depois da refatoração, execute o seguinte comando: *start "metrics-after-pytest\coverage_depois_html\index.html"*. O gráfico precisa ter um bom contraste para facilitar a visualização.

File	Statements				Branches			Total
	coverage	statements	missing	excluded	coverage	branches	partial	
extract_metrics_after_codecarbon.py	0%	28	28	0	0%	2	0	0%
extract_metrics_after_pylint.py	0%	58	58	0	0%	8	0	0%
extract_metrics_after_pytest.py	0%	32	32	0	0%	8	0	0%
extract_metrics_after_radon.py	0%	111	111	0	0%	34	0	0%
extract_metrics_before_codecarbon.py	0%	28	28	0	0%	2	0	0%
extract_metrics_before_pylint.py	0%	58	58	0	0%	8	0	0%
extract_metrics_before_pytest.py	0%	32	32	0	0%	8	0	0%
extract_metrics_before_radon.py	0%	111	111	0	0%	34	0	0%
extract_score_after_pylint.py	0%	13	13	0	0%	4	0	0%
extract_score_before_pylint.py	0%	13	13	0	0%	4	0	0%
imbens__init__.py	100%	11	0	0	100%	0	0	100%
imbens\version.py	100%	1	0	0	100%	0	0	100%
imbens\base.py	100%	4	0	0	100%	0	0	100%
imbens\datasets__init__.py	100%	5	0	0	100%	0	0	100%
imbens\datasets\imbalance.py	41%	22	13	0	0%	6	0	32%
imbens\datasets\openml.py	21%	103	81	4	0%	24	0	17%
imbens\datasets\zenodo.py	35%	66	43	0	8%	24	0	28%
imbens\datasets\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\datasets\tests\test_imbalance.py	0%	32	32	0	0%	2	0	0%
imbens\datasets\tests\test_zenodo.py	0%	37	37	0	0%	2	0	0%
imbens\ensemble__init__.py	100%	21	0	0	100%	0	0	100%
imbens\ensemble_bagging.py	31%	238	164	9	0%	66	0	24%
imbens\ensemble_boost.py	24%	373	283	8	0%	82	0	20%
imbens\ensemble_compatible__init__.py	100%	3	0	0	100%	0	0	100%
imbens\ensemble_compatible\adaboost_compatible.py	36%	107	68	29	0%	20	0	31%
imbens\ensemble_compatible_bagging_compatible.py	36%	140	89	30	0%	40	0	28%
imbens\ensemble_compatible\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble_compatible\tests\test_adaboost_compatible.py	0%	44	44	0	100%	0	0	0%
imbens\ensemble_compatible\tests\test_bagging_compatible.py	0%	171	171	0	0%	10	0	0%
imbens\ensemble_over_sampling__init__.py	100%	6	0	0	100%	0	0	100%
imbens\ensemble_over_sampling\kmeans_smote_boost.py	59%	37	15	35	0%	2	0	56%
imbens\ensemble_over_sampling\over_bagging.py	69%	36	11	28	100%	0	0	69%
imbens\ensemble_over_sampling\over_boost.py	69%	29	9	32	100%	0	0	69%
imbens\ensemble_over_sampling\smote_bagging.py	61%	36	14	29	0%	2	0	58%
imbens\ensemble_over_sampling\smote_boost.py	61%	36	14	37	0%	2	0	58%
imbens\ensemble_over_sampling\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble_over_sampling\tests\test_kmeans_smote_boost.py	0%	61	61	0	0%	6	0	0%
imbens\ensemble_over_sampling\tests\test_over_bagging.py	0%	172	172	0	0%	10	0	0%
imbens\ensemble_over_sampling\tests\test_over_boost.py	0%	46	46	0	100%	0	0	0%
imbens\ensemble_over_sampling\tests\test_smote_bagging.py	0%	172	172	0	0%	10	0	0%
imbens\ensemble_over_sampling\tests\test_smote_boost.py	0%	61	61	0	0%	6	0	0%
imbens\ensemble_reweighting__init__.py	100%	4	0	0	100%	0	0	100%
imbens\ensemble_reweighting\adacost.py	61%	36	14	35	0%	2	0	58%
imbens\ensemble_reweighting\adauboot.py	38%	69	43	35	0%	18	0	30%
imbens\ensemble_reweighting\asymmetric_boost.py	58%	38	16	31	0%	4	0	52%
imbens\ensemble_reweighting\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble_reweighting\tests\test_adacost.py	0%	77	77	0	0%	10	0	0%
imbens\ensemble_reweighting\tests\test_adauboot.py	0%	78	78	0	0%	10	0	0%
imbens\ensemble_reweighting\tests\test_asymmetric_boost.py	0%	80	80	0	0%	12	0	0%
imbens\ensemble_under_sampling__init__.py	100%	7	0	0	100%	0	0	100%
imbens\ensemble_under_sampling\balance_cascade.py	33%	89	60	30	0%	10	0	29%
imbens\ensemble_under_sampling\balanced_random_forest.py	35%	240	157	37	0%	68	0	27%
imbens\ensemble_under_sampling\easy_ensemble.py	59%	37	15	33	0%	4	0	54%
imbens\ensemble_under_sampling\rus_boost.py	68%	31	10	33	100%	0	0	68%
imbens\ensemble_under_sampling\self_paced_ensemble.py	34%	85	56	39	0%	8	0	31%
imbens\ensemble_under_sampling\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble_under_sampling\tests\test_balance_cascade.py	0%	98	98	0	0%	30	0	0%
imbens\ensemble_under_sampling\tests\test_balanced_random_forest.py	0%	97	97	0	0%	2	0	0%
imbens\ensemble_under_sampling\tests\test_easy_ensemble.py	0%	190	190	0	0%	12	0	0%
imbens\ensemble_under_sampling\tests\test_rus_boost.py	0%	46	46	0	100%	0	0	0%
imbens\ensemble_under_sampling\tests\test_self_paced_ensemble.py	0%	79	79	0	0%	16	0	0%
imbens\ensemble_under_sampling\tests\test_under_bagging.py	0%	172	172	0	0%	10	0	0%
imbens\ensemble_under_sampling\tests\test_under_boost.py	69%	29	9	28	100%	0	0	69%
imbens\ensemble\base.py	27%	274	201	50	0%	78	0	21%
imbens\ensemble\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\ensemble\tests\test_base_ensemble.py	0%	41	41	0	0%	4	0	0%
imbens\exceptions.py	50%	2	1	0	100%	0	0	50%
imbens\metrics__init__.py	100%	9	0	0	100%	0	0	100%
imbens\metrics_classification.py	97%	236	6	5	86%	78	11	95%
imbens\metrics\pairwise.py	100%	45	0	0	100%	16	0	100%
imbens\metrics\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\metrics\tests\test_classification.py	100%	189	0	0	100%	10	0	100%

imbens\metrics\tests\test_pairwise.py	100%	100	0	0	100%	8	0	100%
imbens\metrics\tests\test_score_objects.py	100%	28	0	0	100%	0	0	100%
imbens\pipeline.py	16%	118	99	15	0%	48	0	11%
imbens\sampler__init__.py	100%	22	0	0	100%	0	0	100%
imbens\sampler\over_sampling__init__.py	100%	7	0	0	100%	0	0	100%
imbens\sampler\over_sampling\adasy.py	24%	70	53	29	0%	14	0	20%
imbens\sampler\over_sampling\random_over_sampler.py	27%	98	72	19	0%	28	0	21%
imbens\sampler\over_sampling\smote__init__.py	100%	5	0	0	100%	0	0	100%
imbens\sampler\over_sampling\smote\base.py	30%	76	53	19	0%	14	0	26%
imbens\sampler\over_sampling\smote\tests\test_cluster.py	26%	103	76	13	0%	28	0	21%
imbens\sampler\over_sampling\smote\filter.py	21%	127	100	9	0%	36	0	17%
imbens\sampler\over_sampling\smote\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\sampler\over_sampling\smote\tests\test_borderline_smote.py	0%	23	23	0	100%	0	0	0%
imbens\sampler\over_sampling\smote\tests\test_kmeans_smote.py	0%	54	54	0	100%	0	0	0%
imbens\sampler\over_sampling\smote\tests\test_smote.py	0%	40	40	0	100%	0	0	0%
imbens\sampler\over_sampling\smote\tests\test_sym_smote.py	0%	29	29	0	100%	0	0	0%
imbens\sampler\over_sampling\base.py	100%	6	0	4	100%	0	0	100%
imbens\sampler\over_sampling\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\sampler\over_sampling\tests\adasy.py	0%	34	34	0	100%	0	0	0%
imbens\sampler\over_sampling\tests\test_random_over_sampler.py	0%	129	129	0	0%	10	0	0%
imbens\sampler__under_sampling__init__.py	100%	14	0	0	100%	0	0	100%
imbens\sampler__under_sampling\prototype_generation__init__.py	100%	2	0	0	100%	0	0	100%
imbens\sampler__under_sampling\prototype_generation\cluster_centroids.py	29%	65	46	24	0%	22	0	22%
imbens\sampler__under_sampling\prototype_generation\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\sampler__under_sampling\prototype_generation\tests\test_cluster_centroids.py	0%	70	70	0	0%	2	0	0%
imbens\sampler__under_sampling\prototype_selection__init__.py	100%	13	0	0	100%	0	0	100%
imbens\sampler__under_sampling\prototype_selection\balance_cascade_under_sampler.py	28%	68	49	43	0%	8	0	25%
imbens\sampler__under_sampling\prototype_selection\condensed_nearest_neighbour.py	25%	67	50	25	0%	20	0	20%
imbens\sampler__under_sampling\prototype_selection\edited_nearest_neighbours.py	22%	129	101	46	0%	32	0	17%
imbens\sampler__under_sampling\prototype_selection\instance_hardness_threshold.py	38%	48	30	24	0%	10	0	31%
imbens\sampler__under_sampling\prototype_selection\nearmiss.py	21%	75	59	29	0%	24	0	16%
imbens\sampler__under_sampling\prototype_selection\neighbourhood_cleaning_rule.py	27%	62	45	24	0%	10	0	24%
imbens\sampler__under_sampling\prototype_selection\one_sided_selection.py	27%	62	45	25	0%	12	0	23%
imbens\sampler__under_sampling\prototype_selection\random_under_sampler.py	27%	55	40	31	0%	14	0	22%
imbens\sampler__under_sampling\prototype_selection\self_paced_under_sampler.py	18%	122	100	38	0%	42	0	13%
imbens\sampler__under_sampling\prototype_selection\tomek_links.py	42%	36	21	23	0%	10	0	33%
imbens\sampler__under_sampling\prototype_selection\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\sampler__under_sampling\prototype_selection\tests\test_allknn.py	0%	43	43	0	100%	0	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_balance_cascade_under_sampler.py	0%	69	69	0	0%	18	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_condensed_nearest_neighbour.py	0%	36	36	0	100%	0	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_edited_nearest_neighbours.py	0%	47	47	0	100%	0	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_instance_hardness_threshold.py	0%	49	49	0	0%	4	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_nearmiss.py	0%	41	41	0	0%	6	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_neighbourhood_cleaning_rule.py	0%	25	25	0	100%	0	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_one_sided_selection.py	0%	38	38	0	100%	0	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_random_under_sampler.py	0%	62	62	0	0%	4	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_repeated_edited_nearest_neighbours.py	0%	52	52	0	100%	0	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_self_paced_under_sampler.py	0%	76	76	0	0%	16	0	0%
imbens\sampler__under_sampling\prototype_selection\tests\test_tomek_links.py	0%	15	15	0	100%	0	0	0%
imbens\sampler__under_sampling\base.py	100%	9	0	4	100%	0	0	100%
imbens\sampler\base.py	31%	77	53	37	0%	12	0	27%
imbens\sampler\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\sampler\tests\test_base_sampler.py	0%	62	62	0	0%	16	0	0%
imbens\utils__init__.py	100%	10	0	0	100%	0	0	100%
imbens\utils\docstring.py	91%	118	11	25	79%	52	9	87%
imbens\utils\evaluate.py	43%	14	8	4	0%	6	0	30%
imbens\utils\plot.py	0%	56	56	6	0%	20	0	0%
imbens\utils\show_versions.py	0%	30	30	0	0%	12	0	0%
imbens\utils\validation_data.py	21%	43	34	0	0%	22	0	14%
imbens\utils\validation_param.py	17%	331	275	0	0%	182	0	11%
imbens\utils\validation.py	23%	220	169	4	6%	108	1	18%
imbens\utils\deprecation.py	0%	10	10	0	0%	6	0	0%
imbens\utils\estimator_checks.py	0%	238	238	0	0%	40	0	0%
imbens\utils\testing.py	0%	46	46	0	0%	18	0	0%
imbens\utils\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\utils\tests\test_deprecation.py	0%	11	11	0	100%	0	0	0%
imbens\utils\tests\test_docstring.py	0%	18	18	0	100%	0	0	0%
imbens\utils\tests\test_estimator_checks.py	0%	86	86	0	0%	10	0	0%
imbens\utils\tests\test_plot.py	0%	10	10	0	100%	0	0	0%
imbens\utils\tests\test_show_versions.py	0%	49	49	0	100%	0	0	0%
imbens\utils\tests\test_testing.py	0%	13	13	0	0%	2	0	0%
imbens\utils\tests\test_validation_param.py	0%	89	89	0	0%	40	0	0%
imbens\utils\tests\test_validation.py	0%	172	172	0	0%	2	0	0%
imbens\visualizer__init__.py	100%	2	0	0	100%	0	0	100%
imbens\visualizer\tests__init__.py	100%	0	0	0	100%	0	0	100%
imbens\visualizer\tests\test_visualizer.py	0%	41	41	0	0%	2	0	0%
imbens\visualizer\visualizer.py	17%	329	272	5	0%	102	0	13%
script_Daniel.py	0%	7	7	0	0%	2	0	0%
setun.py	0%	28	28	0	100%	0	0	0%

4.4.1 Testes Passaram vs Testes Falharam Após a Refatoração

Aqui você deve informar quantos testes falharam e quantos testes passaram depois da refatoração. Para isso, basta visualizar a página HTML `pytest_depois.html`. Para acessá-la, execute o seguinte comando: `start "metrics-after-pytest\pytest_depois.html"`

Quantidade de testes que passaram: 198

Quantidade de testes que falharam: 0

5. Comparação dos Resultados

5.2.1 Comparação da Complexidade Ciclométrica por Arquivo

Após a refatoração, houve mudança nos arquivos com maior complexidade. A maior soma de complexidade passou de **69** para **138**, enquanto a complexidade máxima permaneceu entre **7 e 16**. Os arquivos com melhores métricas continuaram classificados como **Rank A**, mantendo baixa complexidade.

5.2.2 Comparação da Complexidade Ciclométrica por Função

A maior complexidade por função aumentou de **16** para **21**, surgindo uma função classificada como **Rank D**. As demais funções críticas permaneceram entre **16 e 18** (Rank C), enquanto as funções mais simples continuaram com complexidade **1** (Rank A).

5.2.3 Comparação do Índice de Manutenibilidade

O menor índice de manutenibilidade reduziu de **33,78** para **18,56**, fazendo com que um arquivo passasse de **Rank A** para **Rank B**. Apesar disso, os melhores arquivos permaneceram com índice **100,0** e classificação **Rank A**.

5.2.4 Comparação das Métricas Brutas

As métricas brutas aumentaram significativamente após a refatoração. O **LOC** passou de **5.945** para **32.571**, o **LLOC** de **1.854** para **11.089**, o **SLOC** de **2.695** para **20.439**, o número de **comentários** de **370** para **1.655** e as **linhas multilinha** de **1.293** para **5.121**, indicando um crescimento considerável do projeto e de suas funcionalidades.

5.3.2 Comparação das Métricas do CodeCarbon

Após a refatoração, observou-se uma redução significativa nas métricas ambientais. O tempo de execução diminuiu de **105,92 s** para **18,27 s**, enquanto as emissões de carbono passaram de **$9,24 \times 10^{-5}$** para **$1,91 \times 10^{-5}$** . O consumo total de energia também reduziu de **$9,39 \times 10^{-4}$** para **$1,94 \times 10^{-4}$** , indicando uma execução mais eficiente.

5.3.3 Comparação da Distribuição dos Code Smells

A distribuição dos principais *code smells* permaneceu semelhante após a refatoração. O problema mais frequente continuou sendo **duplicate-code**, com uma pequena redução de **491** para **489** ocorrências. Outros *smells* apresentaram pequenas variações, como **too-many-arguments**, que aumentou de **66** para **85**, e **no-else-return**, que reduziu de **55** para **50**. No geral, não houve mudanças significativas na distribuição dos principais problemas apontados pelo Pylint.

5.3.4 Comparação da Distribuição dos Problemas de Código

A distribuição dos problemas de código permaneceu semelhante após a refatoração. Antes, havia **1.722** ocorrências de *Convention* (48,49%), **810** de *Refactor* (22,81%), **752** de *Warning* (21,18%) e **267** de *Error* (7,52%). As categorias continuaram com comportamento semelhante, indicando que a refatoração não alterou significativamente a distribuição dos tipos de problemas identificados pelo Pylint.

5.3.5 Comparação dos Arquivos Mais Críticos

Os arquivos mais críticos permaneceram praticamente os mesmos. O arquivo **visualizer/tests/__init__.py** continuou liderando, com uma pequena redução de **491** para **489** ocorrências. Já **datasets/_openml.py** manteve **171** ocorrências, enquanto **ensemble_boost.py** reduziu de **113** para **103** e **balanced_random_forest.py** de **87** para **106** (com mudanças na distribuição das categorias de problemas). No geral, houve poucas alterações entre os arquivos com maior número de ocorrências.

5.3.6 Comparação do Score do Projeto

O score do Pylint apresentou uma pequena melhora após a refatoração, passando de **5,07/10** para **5,52/10**. Esse aumento indica uma leve melhoria na qualidade do código, embora ainda existam diversos problemas apontados pela ferramenta.

5.4.2 Comparação da Cobertura de Testes

A cobertura de testes apresentou poucas alterações após a refatoração. Alguns arquivos tiveram pequenas melhorias, como `datasets/_openml.py`, que passou de **12%** para **21%**, `ensemble/_bagging.py`, de **25%** para **31%**, `ensemble/base.py`, de **23%** para **27%**, e `visualizer/visualizer.py`, de **15%** para **17%**.

Entretanto, diversos arquivos permaneceram com **0%** de cobertura, enquanto outros continuaram com **100%**, indicando que a cobertura geral do projeto sofreu apenas pequenas variações após a refatoração.

5.4.3 Comparação dos Resultados dos Testes

Os resultados dos testes permaneceram inalterados após a refatoração. Tanto antes quanto depois, **198 testes foram aprovados e nenhum teste falhou**. Isso indica que as alterações realizadas não comprometeram o comportamento esperado do sistema, preservando sua corretude funcional.

6. Percepções sobre a Prática

Aqui você vai descrever com suas palavras as percepções e aprendizados que você obteve durante o desenvolvimento dessa atividade. Cada um dos tópicos abaixo deve ser um parágrafo.

Experiência geral com a atividade: A atividade proporcionou uma experiência prática com análise de métricas, identificação de *code smells* e contribuição para um projeto *open source*. Uma das minhas dificuldades foi o OpenCode, que em alguns momentos não aplicava as alterações corretamente e ainda possuía limite de uso na versão gratuita. Apesar disso, a atividade ajudou a compreender melhor como ferramentas de análise auxiliam na melhoria da qualidade do código.

Code smells: A refatoração dos *code smells* melhora a legibilidade e facilita a manutenção do projeto. Entre os problemas não refatorados, considero o **duplicate-code** o mais importante, pois representava cerca de **60%** das ocorrências encontradas pelo Pylint. Sua redução evitaria retrabalho e facilitaria futuras modificações. Como benefício, a refatoração deixou o código mais organizado, embora as melhorias nas métricas tenham sido pequenas.

Uso da LLM na refatoração: A LLM compreendeu bem os problemas apontados pelo Pylint e forneceu sugestões úteis, acelerando o processo de refatoração, apesar dos problemas de alteração e limite de uso. Acredito que o uso de LLMs pode facilitar bastante a contribuição em projetos *open source*.

Contribuição com projeto open source: Contribuir para um projeto *open source* permitiu conhecer melhor o fluxo de trabalho com Git e GitHub e trabalhar em um projeto real. As maiores dificuldades foram entender a estrutura do projeto e lidar com problemas nas ferramentas. Uma documentação mais completa facilitaria a entrada de novos colaboradores.

7. Links

- Link do fork: <https://github.com/Jeudess/imbalanced-ensemble>
- Link do projeto: <https://github.com/ZhiningLiu1998/imbalanced-ensemble>
- Link do PR 1: <https://github.com/ZhiningLiu1998/imbalanced-ensemble/pull/42>